10

AI Evolving with New Age Techniques

10.1 Fuzzy Logic

The word **'fuzzy'** means the things that are not clear (or are vague). In real-life situations, we often encounter circumstances in which it is difficult to determine whether to go this way or that or given a statement, it is difficult to judge whether it is true or false. In such a situation, fuzzy logic provides many values between the true and false thereby offering the flexibility to find the best solution for the given situation at that particular point in time. As seen from **Fig. 10.1**, fuzzy logic contains multiple logical values that are the truth values of a variable.

In fuzzy logic, instead of saying just hot or cold, we can have multiple values like Certainly Yes, Possibly Yes, Cannot Say, Possibly No and Certainly No.

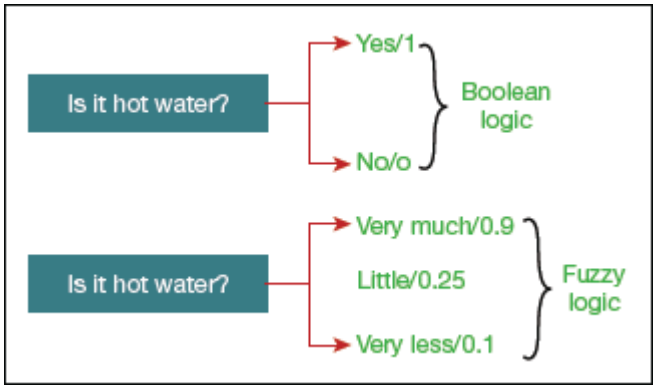


FIGURE 10.1 Example of Fuzzy Logic as compared to Boolean Logic

Fuzzy logic that is based on the **Fuzzy Set Theory** was introduced by **Lofti Zadeh** in **1965. It** provides a range of possibilities that used to be

generated by humans but were not offered by computers. Computer systems followed Boolean system that were limited to only two values- 0 or 1, where 1 denotes the absolute truth value and 0 denotes the absolute false value. Fuzzy system offers multiple possibilities between the 0 and 1, so that the solution can be partially false and partially true. It is a very important concept that is often implemented in micro-controllers, workstation-based or large network-based systems in their hardware or software for achieving the definite output.

10.1.1 Characteristics of Fuzzy Logic

1. It is flexible.
2. It is easy to understand and implement.
3. Fuzzy logic is useful for finding the best solution of problems that have approximate or uncertain reasoning.
4. It is used to develop functions which are non-linear of arbitrary complexity.
5. In fuzzy logic, everything is a matter of degree.
6. It allows any logical system to be easily fuzzified.
7. It is based on natural language processing.
8. It helps to improve the execution time of the algorithms.

10.1.2 Architecture of a Fuzzy Logic System

The architecture of the **fuzzy logic** system as shown in **Fig. 10.2** consists of four components which are discussed below.

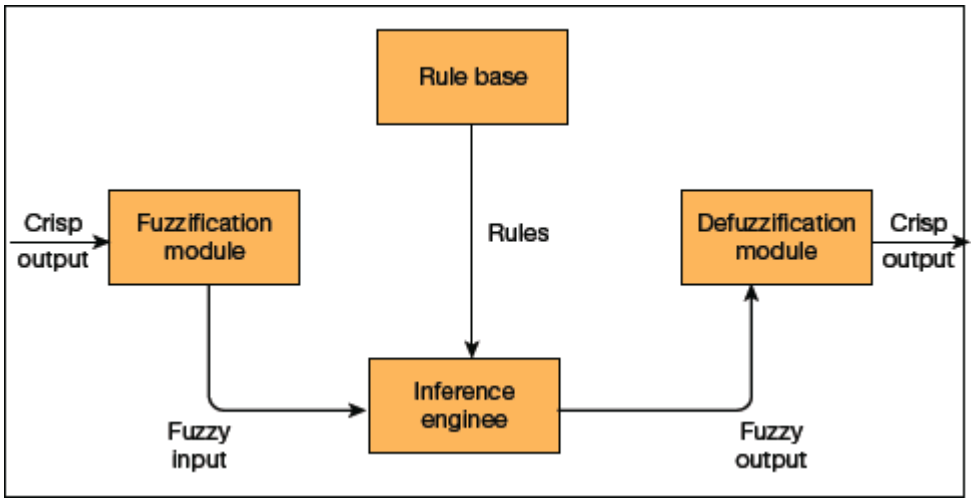


FIGURE 10.2 Architecture of fuzzy logic system

Rule base, as we know, is used for storing rules in the form of If-Then conditions. These conditions are added by the experts and are usually used by the decision-making systems for making vital decisions. Recent advances in Fuzzy theory offer effective methods for designing and tuning of fuzzy controllers. With these updates, the number of rules stored in the rule base has come down considerably.

Fuzzification is used for converting the crisp system inputs into fuzzy inputs. The crisp inputs usually come from sensors but cannot be directly processed by the inference engine. So, they first undergo fuzzification module where the crisp input is divided into following five states in any fuzzy logic system as follows:

1. Large positive (LP)
2. Medium positive (MP)
3. Small (S)
4. Medium negative (MN)
5. Large negative (LN)

Inference engine is the most important component of a fuzzy logic system that processes information. The engine interacts with the rules base to find a rule that is best applicable on the current fuzzy input. Once all rules are fired, the results of their application on fuzzy input are combined to give the control actions.

Defuzzification component accepts fuzzy inputs from the **inference engine** to transform them into a crisp value that is acceptable by the user.

10.1.3 Membership Function

The membership function, also known as indicator or characteristics function, was introduced by Zadeh. The function represents the graph of fuzzy sets to facilitate users to quantify the linguistic term. This graph maps each element of the fuzzy set, x to a value between 0 and 1.

For example, for the Fuzzy set B, the membership function for X is defined as:

$$\mu B: X \rightarrow [0,1]$$

This implies that, in function X, each element of set B is mapped to a value between 0 and 1. This is called a degree of membership or membership value. It quantifies the degree of membership of the element in *X* to the fuzzy set *A*. When plotting these values on a graph, the universal set or the universe of discourse is marked on X-axis and the degrees of membership in the [0, 1] interval is represented on the Y-axis.

There may be multiple membership functions that can fuzzify a numerical value. However, we usually use simple membership functions rather than complex functions as complexity of these functions do not improve the precision in the output.

10.1.4 Example of a Fuzzy Logic System

Let us consider a fuzzy logic system in an air conditioning system having five levels. The system adjusts the temperature of the air conditioner by comparing the room temperature and the target temperature value (as shown in **Fig. 10.3**). To develop such a system, follow the steps given below.

Step 1 – Define linguistic variables and terms

In this system, linguistic variables are input and output variables in the form of simple words or sentences. To specify room temperature, linguis-

tic terms can be cold, warm, hot, etc. So, we can write it as,

$$\text{Temperature (t)} = \{\text{very-cold, cold, warm, very-warm, hot}\}$$

Step 2 – Construct membership functions for them

The membership functions of temperature variable are given in **Fig. 10.4**.

Step 3 – Construct knowledge base rules

In this step, a matrix of room temperature values versus target temperature values that an air conditioning system is expected to provide is created. In the matrix given below, room temperature is written in the leftmost (first) column and the first row of other columns gives the target temperature. Except the cells in the first column and the first row, other cells in the matrix specify the action that needs to be performed.

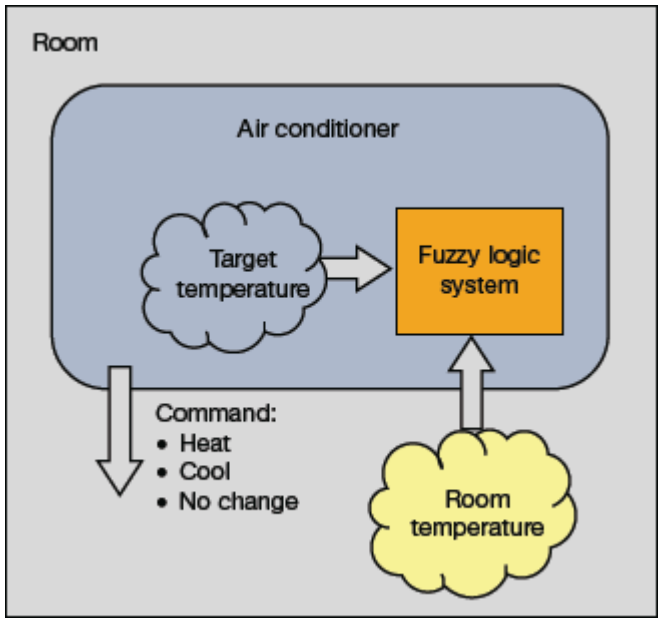


FIGURE 10.3 Fuzzy logic system in an air conditioning system

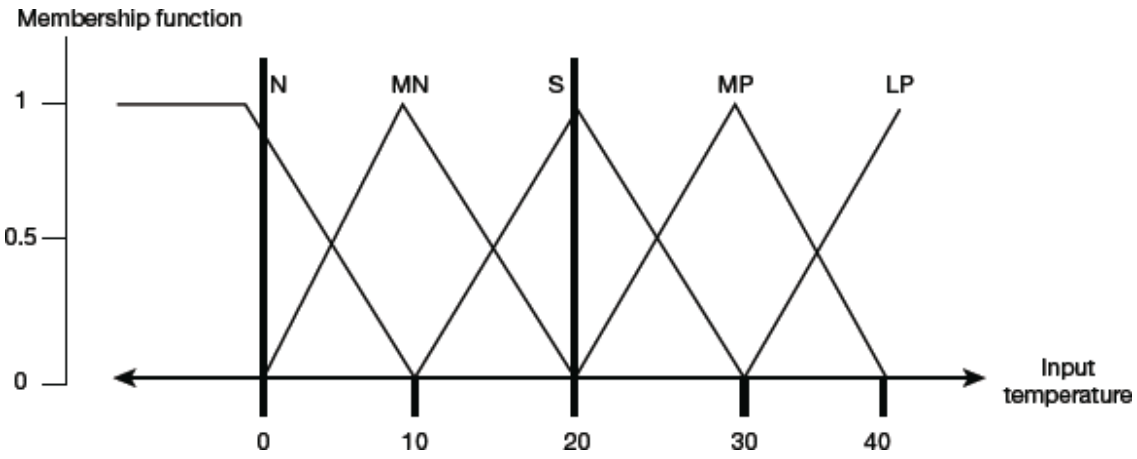


FIGURE 10.4 Membership functions of temperature variable

TABLE 10.1 Knowledge base for Temperature

Roomtemp. /Target	Very_cold	Cold	Warm	Hot	Very_hot
Very_Cold	No_Change	Heat	Heat	Heat	Heat
Cold	Cool	No_Change	Heat	Heat	Heat
Warm	Cool	Cool	No_Change	Heat	Heat
Hot	Cool	Cool	Cool	No_Change	Heat
Very_Hot	Cool	Cool	Cool	Cool	No_Change

For example, the highlighted cell states that if the room temperature is WARM and the expected temperature is HOT, then we need to increase HEATING.

Now, we will create a set of rules into the knowledge base in the form of IF-THEN-ELSE structures as shown in **Table 10.2**.

TABLE 10.2 Creating Rules

Sr. No.	Condition	Action
1	IF temperature = (Cold OR Very_Cold) AND target = Warm THEN	Heat
2	IF temperature = (Hot OR Very_Hot) AND target = Warm THEN	Cool
3	IF (temperature = Warm) AND (target = Warm) THEN	No_Change

Step 4 – Obtain fuzzy value

Fuzzy values are obtained by applying fuzzy set operations on the rules. The final fuzzy value is obtained by combining all results obtained by evaluating the rules.

Step 5 – Perform defuzzification

Defuzzification is performed based on the membership function for output variable as shown in **Fig. 10.5**.

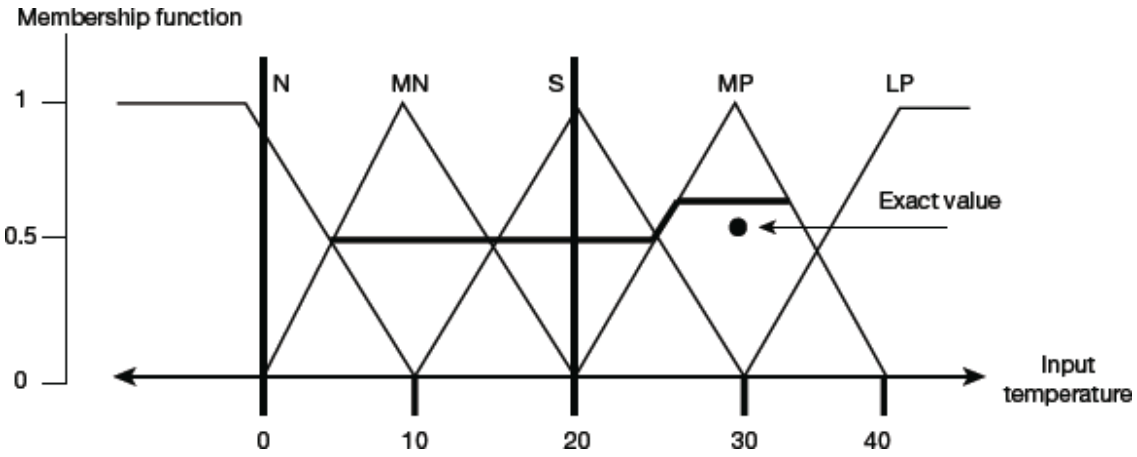


FIGURE 10.5 Membership function for output variable in case of Defuzzification

10.1.5 Classical and Fuzzy Set Theory

Before going into the Fuzzy set theory, let us first quickly revise the set theory which we are all already familiar with.

10.1.5.1 Set Theory

A set is a collection of unordered or ordered elements. For example, we have a set of, all-natural numbers, even numbers, students in a class, rivers in a country, etc. these sets can be classified in various categories as shown in **Fig. 10.6**.

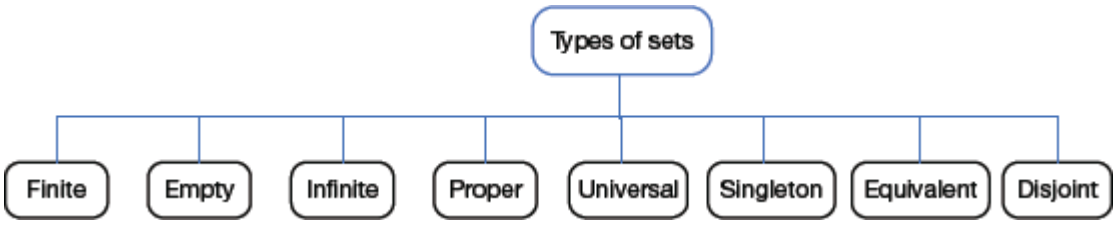


FIGURE 10.6 Set Theory

However, a classical set is a collection of distinct elements and has crisp boundaries. The classical set is defined in such a way that the universe of discourse (set of all possible values, also known as universal set) is divided into two groups—**members** and **non-members**. Therefore, such a set does not support **partial membership.** We can represent a classical set either in roaster form or in set-builder form.

Roaster form, also known as a tabular form, denotes a set by enclosing comma-separated members of the set within the brackets (as given below):

Set_name = {element1, element2, element3, ……, element N}

For example, set of Natural Numbers can be written as: **`N = {1, 2, 3, 4, 5, 6, 7, ……,n)`**.

Similarly, set of Even Numbers less than 20 can be written as,

X = {2, 4, 6, 8, 10, 12, 14, 16, 18}.

Set builder form defines a set by stating the common properties of elements in the set. It can be denoted as:

A = {x : p(x)}

For example, the set **`{2, 4, 6, 8, 10, 12, 14, 16, 18}`** is written as:

B = {x:2 ≤ x < 20 and (x%2) = 0}

10.1.5.2 Operations on Classical Set

We can perform different operations on classical sets like union, intersection, difference and complement.

Union (A U B)

The result of this operation generates a set of those elements which exist in either appear in A or in B or in both. This means that the union operation combines all the elements from both the sets in a new set. For this reason, union operation is also known as a Logical OR operation on the set. It is denoted as follows:

$$A \cup B = \{ x \mid x \in A \text{ OR } x \in B \}.$$

For example, given

Set A = {0, 1, 2, 3}

Set B = {1, 2, 3, 4, 5}, then

A ∪ B = {0, 1, 2, 3, 4, 5}

Intersection (A ∩ B)

The intersection or Logical AND operation returns a new set of those elements which are present in both set A and in set B. The operation is described as, $A \cap B = \{ x \mid x \in A \text{ AND } x \in B \}$.

For example, given

Set A = {0, 1, 2, 3}

Set B = {1, 2, 3, 4, 5}, then

A ∩ B = {1, 2, 3}

Difference Operation (A – B)

The difference operation returns a new set of those elements which exist only in set A but not in set B. It is described as follows:

$$A - B = \{ x \mid x \in A \text{ AND } x \notin B \}$$

For example, given

Set A = {0, 1, 2, 3}

Set B = {2, 3, 4, 5}, then

A - B = {0,1}

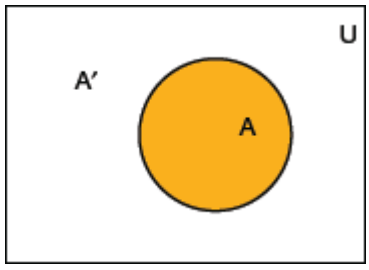
Complement Operation (A')

The complement operation is applied on a single set. It returns a set of elements which do not exist in set A and can be represented as,

$$A' = \{x \in U : x \notin A\}$$

Here, U is the Universal set. Therefore, we can say that **complement of a set** is a subset of the universal set as it has all the elements of the universal set except those that are not present in the given set. In other words, the complement of a set A is the difference between the universal set and set A. that is A' = U − A.

For example, if the universal set consists of all prime numbers up to 25, that is, U = {2, 3, 5, 7, 11, 13, 17, 19, 23} and,



A = {2, 3, 5} then,

A' = (U − A)

= {2, 3, 5, 7, 11, 13, 17, 19, 23} − {2, 3, 5}

= {7, 11, 13, 17, 19, 23}

10.1.5.3 Properties of Classical Set

In this section, we will discuss some basic properties of a classical set.

Commutative Property

The property states that operations on a set can be performed irrespective of order of the sets. That is,

$$A \cup B = B \cup A$$
$$A \cap B = B \cap A$$

For example, given:

A = {1, 2, 3}, and B = {2, 3, 4}, then

A ∪ B = {1, 2, 3, 4} and B ∪ A = {1, 2, 3, 4}

Hence, proved. Similarly, let us prove it in case of intersection.

A ∩ B = {2, 3} and B ∩ A = {2, 3}

Associative Property

Associativity property allows us to perform the operations by grouping the operands and keeping them in similar order.

$$\mathbf{(A \cup B) \cup C = A \cup (B \cup C)}$$
$$\mathbf{(A \cap B) \cap C = A \cap (B \cap C)}$$

For example, given:

A = {1, 2, 3}, B = {2, 3, 4}and C = {5, 6}, then

A ∪ B = {1, 2, 3, 4} and (A ∪ B) ∪ C = {1, 2, 3, 4, 5, 6}

B ∪ C = {2,3,4,5,6} and **A** ∪ **(B** ∪ **C)** = {1, 2, 3, 4, 5, 6}

Hence, proved. Now,

A ∩ B = {2, 3} and (A ∩ B) ∩ C = ϕ

B ∩ C = ϕ. Therefore, A ∩ (B ∩ C) = ϕ

Idempotency Property

It is defined as

$$A \cup A = A$$

$$A \cap A = A$$

For example, given A = {1, 2, 3},

$$A \cup A = \{1, 2, 3\} \cup \{1, 2, 3\} = \{1, 2, 3\}$$

Similarly, A ∩ A = {1, 2, 3} ∩ {1, 2, 3} = {1, 2, 3}

Absorption Property

This property when defined on two sets states that,

$$A \cup (A \cap B) = A$$

$$A \cap (A \cup B) = A$$

For example, given:

A = {1, 2, 3}, B = {2, 3, 4}, then

A ∪ (A ∩ B) = {1, 2, 3} ∪ {2, 3} = {1, 2, 3}

Similarly, A ∩ (A ∪ B) = {1, 2, 3} ∩ { 1, 2, 3, 4} = {1, 2, 3}

Distributive Property

Given three finite sets A, B and C, distributive property is defined as,

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

For example, given:

A = {1, 2, 3}, B = {2, 3, 4} and C = {5, 6}, then

$A \cup (B \cap C) = \{1, 2, 3\} \cup \{\phi\} = \{1, 2, 3\}$ (because $\mathbf{A} \cup \varnothing = \mathbf{A}$)

$(A \cup B) \cap (A \cup C) = \{1, 2, 3, 4\} \cap \{1, 2, 3, 5, 6\} = \{1, 2, 3\}$

Hence, proved. Now,

$A \cap (B \cup C) = \{1, 2, 3\} \cap \{2, 3, 4, 5, 6\} = \{2, 3\}$

$(A \cap B) \cup (A \cap C) = \{2, 3\} \cup \{\phi\} = \{2, 3\}$

Identity Property

For a given finite set A and Universal set X, identity property can be given as,

$$A \cup \{\varphi\} = A$$
$$A \cap X = A$$
$$A \cap \{\varphi\} = \{\varphi\}$$
$$A \cup X = X$$

For example, given $A = \{1, 2, 3\}$ and $X = \{1, 2, 3, 4, 5, 6\}$

$A \cup \{\phi\} = \{1, 2, 3\} \cup \{\phi\} = \{1, 2, 3\}$

$A \cap X = \{1, 2, 3\} \cap \{1, 2, 3, 4, 5, 6\} = \{1, 2, 3\}$

$A \cap \{\phi\} = \{1, 2, 3\} \cap \{\phi\} = \{\phi\}$

$A \cup X = \{1, 2, 3\} \cup \{1, 2, 3, 4, 5, 6\} = \{1, 2, 3, 4, 5, 6\}$

Transitive Property

This property state that for the finite sets A, B and C, if A is a subset of B and B is a subset of C, then A is also a subset of C. This can be written as,

IF $A \subseteq B \subseteq C$, then $A \subseteq C$

For example, given

$A = \{1, 2, 3\}$, $B = \{1, 2, 3, 4, 5\}$ and $C = \{1, 2, 3, 4, 5, 6\}$, then

Since all the elements of Aare a part of set B, A is said to be a subset of B.

Since all the elements of set B are a part of set C, B is said to be a subset of C.

Moreover, we see that A is also a subset of C.

Ivolution Property

The property states that for any finite set A,

$$\overline{\overline{A}} = A$$

For example, given A = {1, 2, 3} and U = {1, 2, 3, 4, 5, 6}

A' = {1, 2, 3, 4, 5, 6} − {1, 2, 3} = {4, 5, 6}

A'' = {1, 2, 3, 4, 5, 6} − {4, 5, 6} = {1, 2, 3}

De Morgan's Law

Given two sets A and B, the De Morgan's law says that,

$$(A \cup B)' = A' \cap B'$$
$$(A \cap B)' = A' \cup B'$$

For example, given;

A = {1, 2, 3}, B = {2, 3, 4} and U = {1, 2, 3, 4, 5, 6}

(A ∪ B)' = {{1, 2, 3} ∪ {2, 3, 4}}' = {1, 2, 3, 4}' = {5, 6}

A' ∩ B' = {4, 5, 6} ∩ {1, 5, 6} = {5, 6}

Hence, proved. Similarly,

(A ∩ B)' = {2, 3}' = {1, 4, 5, 6}

A' ∪ B' = {4, 5, 6} ∪ {1, 5, 6} = {1, 4, 5, 6}

10.1.6 Fuzzy Set

The classical set theory is the subset of Fuzzy set theory. Or we can say that the fuzzy set theory is a generalization of the classical theory of set (i.e., crisp set). Introduced by Lofti A. Zadeh in 1965, the entire fuzzy logic is based on the fuzzy set theory.

A fuzzy set denoted by the tilde (~) character is a collection of values which exist between 0 and 1. In the fuzzy set, there is also a concept of partial membership.

A fuzzy set (Ã) is a pair of U and M, where U is the Universal set and M is the membership function which takes on values in the interval [0, 1]. The universal set (U) is also denoted by Ω or X.

$$\tilde{A} = \{(x, m_{\tilde{A}}(x)) \mid x \in X\}$$

10.1.6.1 Operations on Fuzzy Set

Given two fuzzy sets, and a universal set X, with member functions,

$$m_{\tilde{A}}(x) \text{ and } m_{\tilde{B}}(x)$$

The operations defined on the fuzzy set are as follows:

Union Operation

The union operation of a fuzzy set is written as,

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

For example, given:

A = {(X_1, 0.2), (X_2, 0.4), (X_3, 0.6), (X_4, 0.8)}

B = {(X_1, 0.1), (X_2, 0.5), (X_3, 0.7), (X_4, 0.3)} then,

A U B = {(X_1, 0.2), (X_2, 0.5), (X_3, 0.7), (X_4, 0.8)}

Justification:

For X_1	For X_2	For X_3	For X_4
$\mu_{A\cup B}(X_1) = \max(\mu_A(X_1), \mu_B(X_1))$	$\mu_{A\cup B}(X_2) = \max(\mu_A(X_2), \mu_B(X_2))$	$\mu_{A\cup B}(X_3) = \max(\mu_A(X_3), \mu_B(X_3))$	$\mu_{A\cup B}(X_4) = \max(\mu_A(X_4), \mu_B(X_4))$
$\mu_{A\cup B}(X_1) = \max(0.2,0.1)$	$\mu_{A\cup B}(X_2) = \max(0.4,0.5)$	$\mu_{A\cup B}(X_3) = \max(0.6,0.7)$	$\mu_{A\cup B}(X_4) = \max(0.8,0.3)$
$\mu_{A\cup B}(X_1) = 0.2$	$\mu_{A\cup B}(X_2) = 0.5$	$\mu_{A\cup B}(X_3) = 0.7$	$\mu_{A\cup B}(X_4) = 0.8$

Intersection Operation

The intersection operation of a fuzzy set is written as,

$$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

For example, given:

$A = \{(X_1, 0.2), (X_2, 0.4), (X_3, 0.6), (X_4, 0.8)\}$

$B = \{(X_1, 0.1), (X_2, 0.5), (X_3, 0.7), (X_4, 0.1)\}$ then,

$A \cap B = \{(X_1, 0.1), (X_2, 0.4), (X_3, 0.6), (X_4, 0.1)\}$

Justification:

For X_1	For X_2	For X_3	For X_4
$\mu_{A\cap B}(X_1) = \min(\mu_A(X_1), \mu_B(X_1))$	$\mu_{A\cap B}(X_2) = \min(\mu_A(X_2), \mu_B(X_2))$	$\mu_{A\cap B}(X_3) = \min(\mu_A(X_3), \mu_B(X_3))$	$\mu_{A\cap B}(X_4) = \min(\mu_A(X_4), \mu_B(X_4))$
$\mu_{A\cap B}(X_1) = \min(0.2,0.1)$	$\mu_{A\cap B}(X_2) = \min(0.4,0.5)$	$\mu_{A\cap B}(X_3) = \min(0.6,0.7)$	$\mu_{A\cap B}(X_4) = \min(0.8,0.1)$
$\mu_{A\cap B}(X_1) = 0.1$	$\mu_{A\cap B}(X_2) = 0.4$	$\mu_{A\cap B}(X_3) = 0.6$	$\mu_{A\cap B}(X_4) = 0.1$

Complement Operation

The complement operation of fuzzy set is defined by:

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x),$$

For example, given:

$A = \{(X_1, 0.5), (X_2, 0.4), (X_3, 0.2), (X_4, 0.1)\}$, then

$\bar{A} = \{(X_1, 0.5), (X_2, 0.6), (X_3, 0.8), (X_4, 0.9)\}$

Justification:

For X_1	For X_3	For X_4
$\mu_{\bar{A}}(X_1) = 1 - \mu_A(X_1)$	$\mu_{\bar{A}}(X_1) = 1 - \mu_A(X_1)$	$\mu_{\bar{A}}(X_1) = 1 - \mu_A(X_1)$
$\mu_{\bar{A}}(X_1) = 1 - 0.5$	$\mu_{\bar{A}}(X_1) = 1 - 0.2$	$\mu_{\bar{A}}(X_1) = 1 - 0.1$
$\mu_{\bar{A}}(X_1) = 0.5$	$\mu_{\bar{A}}(X_1) = 0.8$	$\mu_{\bar{A}}(X_1) = 0.9$

TABLE 10.3 Differences between classical set theory and fuzzy set theory

Classical Set Theory	Fuzzy Set Theory
This theory is applicable on sets having sharp boundaries.	This theory is applicable on sets that do not have sharp boundaries.
This set theory is defined by exact boundaries that can be either 0 or 1	This set theory is defined by ambiguous boundaries.
There is no uncertainty about the boundary's location of a set.	There is an uncertainty about the boundary's location of a set.
This theory is used in the design of digital systems.	It is mainly used for designing fuzzy controllers.

10.1.7 Applications of Fuzzy Logic

Fuzzy logic is widely used in different application areas in **businesses** for making vital decisions.

Automative systems apply fuzzy logic to control brakes in hazardous cases depending on the speed, acceleration, wheel speed and acceleration of the car. It is also used to select the gear based on engine load, driving style and road conditions.

In auto-transmission systems, fuzzy logic is used to improve the efficiency by controlling the fuel injection and ignition based on throttle setting, cooling water temperature, RPM, etc.

Systems developed for defence use fuzzy logic for underwater target recognition and the automatic target recognition of thermal infrared images.

Fuzzy logic can be used to search fuzzy images.

Pattern recognition and classification systems use fuzzy logic for handwriting recognition.

In **microwave ovens,** fuzzy logic is used to set the lunes power and cooking strategy.

M**odern control systems** like expert systems often use fuzzy logic.

Fuzzy logic is widely used to predict the stock market and for managing funds.

In chemical industry, fuzzy logic is used to control the ph and chemical distillation process.

In manufacturing industry, fuzzy logic used for optimizing production.

In devices like vacuum cleaners, washing machines, heaters, air conditioners and humidifiers, fuzzy logic is used for controlling the timing of events. For example, an elevator uses fuzzy logic to reduce waiting for time-based on passenger traffic.

In a dish-washer, fuzzy logic is used for adjusting the cleaning cycle, rinse and wash strategies depending on the number of dishes and the amount of food served on the dishes.

Copy machine by Canon uses fuzzy logic for adjusting drum voltage based on picture density, humidity and temperature.

In a golf diagnostic system, fuzzy logic selects golf club based on golfer’s swing and physique.

In a kiln, fuzzy logic is used to control the mixing of cement.

10.1.8 Advantages of Fuzzy Logic

IBM’s Watson is a well-known AI system that uses variations of fuzzy logic and fuzzy semantics.

1. Fuzzy logic works similar to human reasoning.
2. The concept is easy to understand.
3. It does not need large memory as the algorithms can work well with fewer data.
4. Fuzzy logic is easily applicable in diverse fields.
5. It provides effective solutions to even highly complex problems.
6. Fuzzy set theory is based on the set theory of mathematics, so users can easily relate to its basic concepts.
7. It facilitates controlling micro-controllers and thus electronic devices.
8. Developing a system based on fuzzy logic takes less time than developing it using conventional methods.
9. Fuzzy logic is very flexible as it allows users to easily add and delete rules in the system.
10. Fuzzy logic supports the concept of partial truth, such as 0.9 or 0.5.
11. Fuzzy logic mimics real-life circumstances, where we rarely have statements that are absolutely true or absolutely false.
12. Fuzzy logic, when used by quantitative analysts, improves the execution of algorithms.
13. Algorithms based on fuzzy logic produce accurate results even with imprecise or inaccurate data.
14. Systems based on fuzzy logic are more robust as they are not dependent on precise inputs.
15. Given the information about how far the subject in a photograph is, a fuzzy logic system can focus the lens on it.

10.1.9 Disadvantages of Fuzzy Logic

1. The execution time of fuzzy logic systems is more than conventional systems, so these systems are often slow to generate output.
2. Output generated by fuzzy logic system is not 100% accurate.
3. In fuzzy logic system, there are multiple ways to solve a given problem. However, this may lead to ambiguity.
4. Problems that need highly accurate solutions should not be solved using fuzzy logic.
5. A fuzzy logic system needs a lot of testing for verification and validation.

6. The accuracy of results obtained from a fuzzy logic system depends on human expertise and knowledge.
7. The design of a fuzzy logic system is understandable only when it is simple.
8. Fuzzy systems lack the capability of machine learning and neural network type pattern recognition.

You must not use fuzzy logic in the following cases:

1. If it is difficult to map an input space to an output space
2. When application of common sense can easily solve the problem
3. Controllers work well without the use of fuzzy logic

10.1.10 Trends

Nowadays, the trend is to use fuzzy logic along with neurocomputing and genetic algorithms. A combination of all these techniques forms the basis of *soft computing. In soft computing, the goal is to* exploit the tolerance for imprecision, uncertainty and partial truth to achieve tractability, robustness and low solution cost. At present, despite of the fact that soft computing systems are expensive, they are being preferred as they build AI system with MIQ (Machine IQ) much higher than that of conventional neuro-fuzzy systems which combine fuzzy logic and neurocomputing and induce rules from observations. An effective method developed by Dr Roger Jang for this purpose is called ANFIS (Adaptive Neuro-Fuzzy Inference System).

Fuzzy logic approximates human reasoning and balances the tradeoff between precision and significance (illustrated in **Fig. 10.7**). For example, when there is an unexpected alert of a mass falling on you, the alert need not be precise about the exact mass and speed as is not at all necessary to make a quick decision.

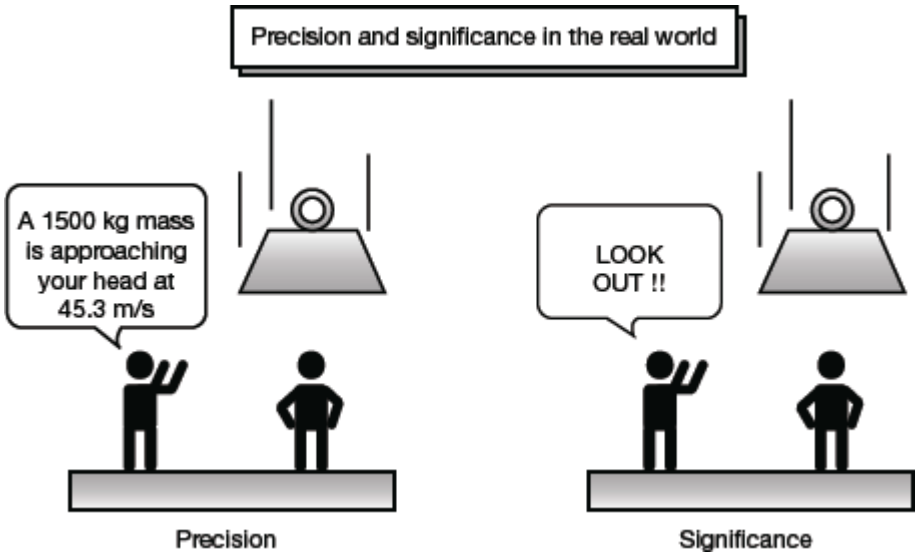


FIGURE 10.7

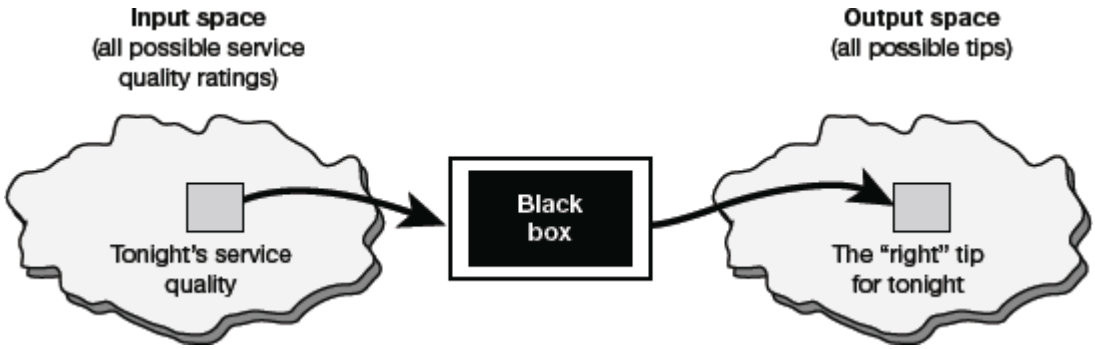


FIGURE 10.8 An input/output map for the tipping problem: "Given the quality of service, how much should I tip?"

A fuzzy system behaves like a black box that maps an input space to an output space. Consider the example given in **Fig. 10.8** where fuzzy logic is used to determine the appropriate amount of tip by mapping the input space of all possible restaurant service ratings to all possible tip values. Note that between the input and the output, there is a black box that may contain fuzzy systems, linear systems, expert systems, neural networks, differential equations, interpolated multidimensional lookup tables or any other algorithm.

10.2 Genetic Algorithms

Nature has always been a great source of inspiration to all mankind. Genetic algorithm is a subset of **evolutionary computation**. It is a search-based optimization technique that draws its concept from the

principles of **genetics and natural selection.** The main take away of these algorithms is that they can find optimal or near-optimal solutions to difficult optimization problems in research, and in machine learning problems. Genetic algorithms can find solutions to those problems which otherwise would take a lifetime to solve.

The term ‘optimization’ refers to the process of **making something better**. For this, the algorithm needs to find what input values will give the ‘best’ output values. Though, what ‘best’ means will vary from problem to problem, mathematically, it refers to maximizing or minimizing one or more objective functions, by varying the input values.

We know that the search space comprises all possible solutions or values which the inputs can take. The problem of optimization is to find those solutions or values in the search space that gives the optimal solution.

In genetic algorithms, the given problem has a **pool or a population of possible solutions**. Crossover and mutation are applied on these solutions to produce new children. This process is repeated over several generations. Centred around the Darwinian Theory of ‘Survival of the Fittest’, each individual (or candidate solution) thus produced is assigned a fitness value. Individuals having a higher fitness value are given a priority to mate and yield more ‘fitter’ individuals. This helps in the process of evolution that gives better solutions over generations, till stopping criterion is met.

Though genetic algorithms are randomized, they perform much better than random local search as they exploit historical information as well.

10.2.1 Advantages of Genetic Algorithms

1. In real-world problems, derivative information may not be available. Genetic algorithms work well without this information.

   Derivative information means any notes, summaries, evaluations, analyses, **as well** as feedback, suggestions, improvements and other material derived by the Restricted Party Group from any of the confidential information.

2. Compared to traditional algorithms, genetic algorithms are fast and efficient. For example, some difficult problems like the Travelling Salesperson Problem (TSP) are widely applied to solve complex real-world problems like path finding, GPS Navigation system and VLSI Design. Delay in computing the 'optimal' path from the source to destination is not acceptable. Genetic algorithms come to rescue in such situations to get optimal results very quickly.
3. They can perform parallel computations.
4. They optimize continuous as well as discrete functions.
5. They give optimized solutions for multi-objective problems.
6. Instead of returning a single 'good' solution, it returns a list of all 'good' solutions.
7. They always find solution to a problem and this solution improves over time.
8. They are very helpful in problems that have a very large search space and involve a large number of parameters.
9. They can be used for solving NP Hard problems. There are a large set of NP Hard problems and they give optimal or near—optimal solutions for these problems in a reasonable amount of time.

   NP Hard problems are the most powerful computing problems that take a very long time (even years!) to solve a problem.

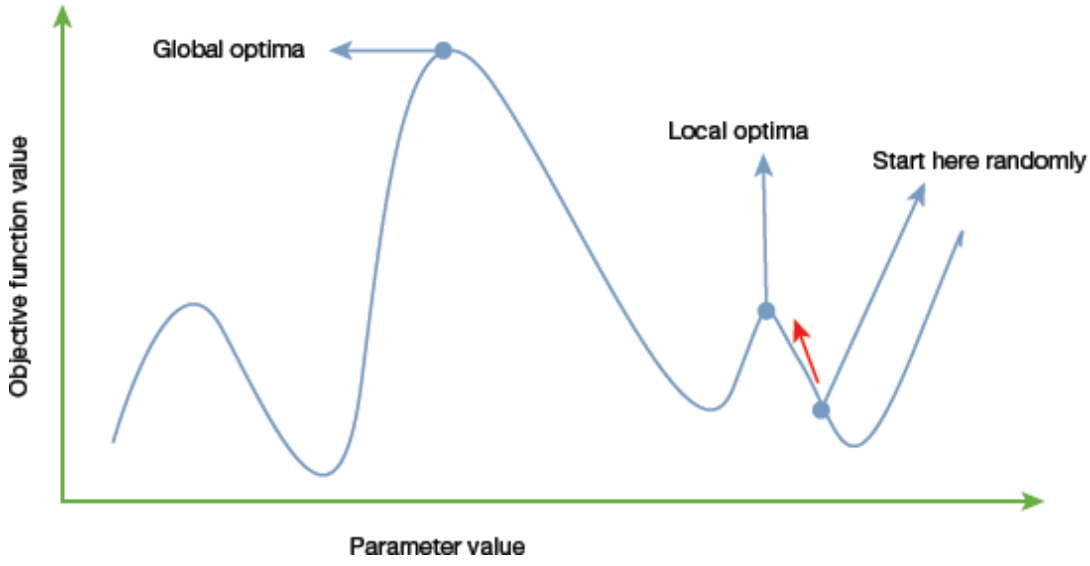


FIGURE 10.9 Global and Local Optima

10. Traditional calculus-based mathematical techniques (like gradient-based methods) start at a random point and move in the direction of the gradient until the top of the hill is reached. This technique is effi-

cient and works very well for functions having a single-peak. For example, the cost function in linear regression can easily be solved using gradient-based methods. However, in most real-world problems, there are many peaks and valleys. These problems cannot be solved using the traditional methods as they end-up being stuck at the local optima (refer to **Fig. 10.9**).

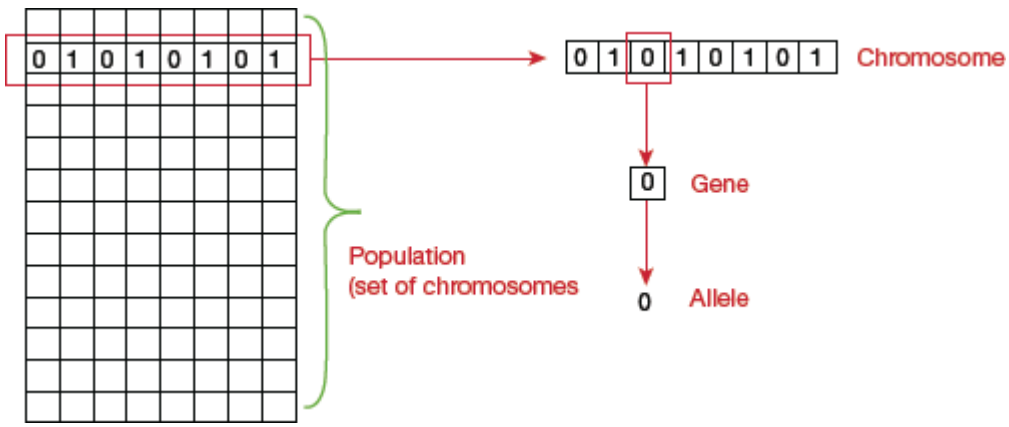
10.2.2 Limitations of Genetic Algorithms

1. Genetic algorithms are not suited for all problems such as those that are either very simple or those for which derivative information is available.
2. The algorithm is computationally expensive for some problems as fitness value is calculated repeatedly.
3. There is no guarantee on the optimality or the quality of the solution.
4. Genetic algorithms may not find the optimal solution if not implemented properly.

10.2.3 Basic Terminology

To understand genetic algorithms, let us familiarize with some basic terminology that will be frequently used.

1. **Population:** It is a subset of all the possible solutions to the given problem.
2. **Chromosomes:** A chromosome is the individual solution to the given problem.
3. **Gene:** A gene is one element position of a chromosome.
4. **Allele:** It is the value which a gene takes for a particular chromosome.



5. **Genotype:** Genotype is the population (solution) in the computation space that can be represented in a simple and easy-to-understand manner so that it can be easily manipulated using a computing system.
6. **Phenotype:** Phenotype is the population (solution) in the real-world solution space that can be represented in real world situations (refer **Fig. 10.10**).
7. **Decoding and Encoding:** For simple problems, the **phenotype and genotype** spaces are the same. But for complex, real-world problems, these spaces are different. While decoding is the process of transforming a solution from the genotype to phenotype space, encoding, on the other hand, transforms solution from the phenotype to genotype space. A genetic algorithm must perform decoding at a very fast pace as it has to be done repeatedly while calculating the fitness value.
   For example, in the 0/1 Knapsack problem, the phenotype space consists of solutions which has the item numbers of the items to be picked. But the genotype space is represented as a binary string of length n (where n is the number of items). A **0 at position x** indicates that the $\mathbf{x^{th}}$ item is not picked, whereas a 1 indicates that it was picked. Thus, in this case, genotype space is different from that of the phenotype.

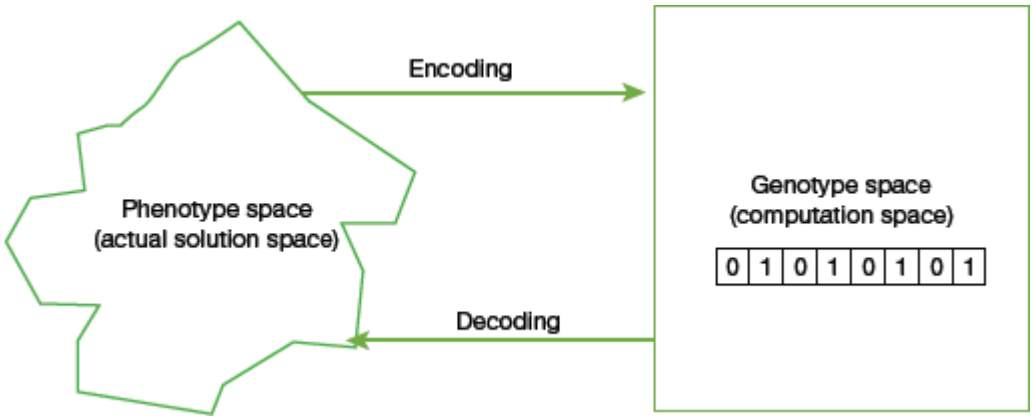


FIGURE 10.10 Genotype and Phenotype Space

8. **Fitness Function** (or **Evaluation Function):** It is the function which accepts the solution as input and produces the suitability of the solution as the output. That is, the f**itness function** determines how fit a solution is. This is done by evaluating how close a given solution is to the optimum solution of the desired problem.

In some cases, the fitness function and the objective function may be the same but usually they both are different. While the objective function is the function that has to be optimized, fitness function, on the other hand, is the function which guides the optimization.

9. **Genetic Operators:** Operators which change the genetic composition of the offspring are known as genetic operators. These include crossover, mutation, selection, etc.

10.2.4 Basic Structure

The basic structure of a genetic algorithm can be given as follows.

Step 1: Start with an initial population. The population may either be generated randomly or seeded by using heuristics.

Step 2: Repeat steps 3 and 4 until the specified condition is satisfied.

Step 3: Select parents from this population for producing new offspring. This is done by applying genetic operators like crossover and mutation.

Step 4: Off-springs generated in Step 3 replace the existing individuals in the population.

In this way genetic algorithms mimic the human evolution to some extent.

10.2.5 Genotype Representation

To implement a genetic algorithm (refer **Fig. 10.11**), we need to first decide how the solutions will be represented. A genetic algorithm can give poor performance if the representation is not correct or inefficient.

Therefore, choosing a proper representation that accurately defines a mapping between the phenotype and genotype spaces is essential for the success of a genetic algorithm.

In this section, we will discuss commonly used representations for genetic algorithms. These representations are problem specific and a mix of these representations is usually used for better results.

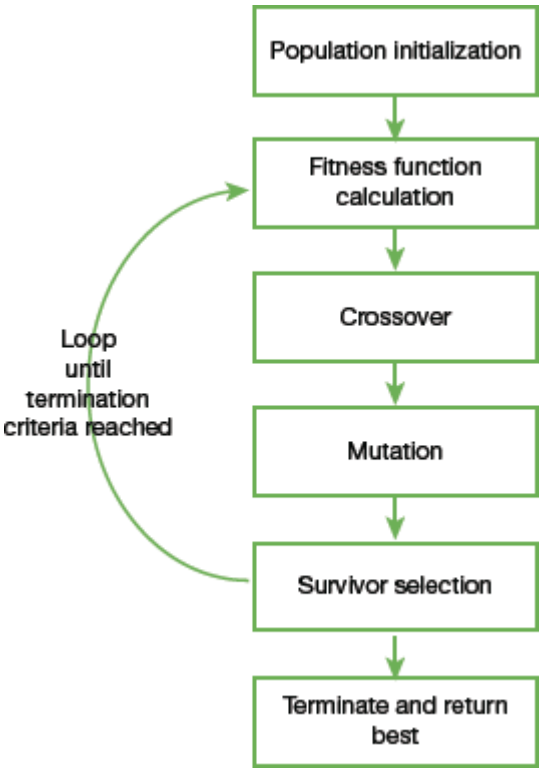
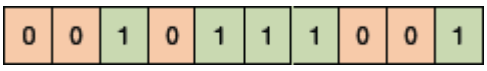


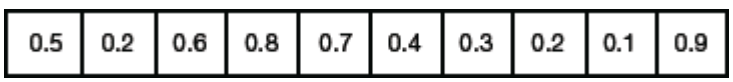
FIGURE 10.11 Steps in a genetic algorithm

Binary Representation

It is one of the simplest and most widely used representation in genetic algorithms. In this representation, the genotype consists of bit strings. Binary representation is best-suited for problems in which the solution space consists of Boolean decision variables—yes or no. For example, in the 0/1 Knapsack problem containing n items, a solution is represented by a binary string of n elements. The value at position is x is 1 if it was picked and 0, otherwise.

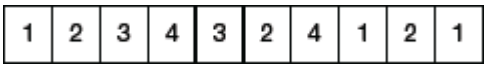


For problems that deal with numbers, we can represent the numbers with their binary representation. However, in this encoding, different bits have different significance. Therefore, mutation and crossover operators may have undesired consequences. This can be resolved to some extent by using gray coding.



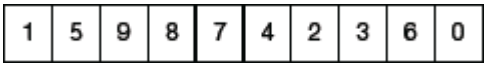
Real Valued Representation

Real-valued representation is used for representing problems in which genes must have continuous rather than discrete values.



Integer Representation

In some problems, the value of genes cannot be restricted to ‘yes’ or ‘no’, ‘0’ or ‘1’. For discrete valued genes, that can have multiple integer values, integer representation is a preferred choice. For example, four directions —north, south, east and west can be encoded as {0, 1, 2, 3}.



Permutation Representation

Permutation representation is best-suited for problems in which the solution is represented by an order of elements. For example, in the travelling salesman problem (TSP), the salesman has to take a tour of all the cities, visit each city exactly once and come back to the city from where he started. This problem is an optimization problem as the total distance to be travelled has to be minimized. The solution to this problem is an ordering or permutation of all the cities. Hence, permutation representation is used for solving such problems.

10.2.6 Population

As discussed, population is a subset of solutions in the current generation. Every individual in population is called chromosome. So, population can be thought of as a set of chromosomes. In biology, we have read that the population must have diversity to support existence. Similarly, in genetic algorithms, diversity of the population should be maintained to avoid premature convergence.

The size of the population should be chosen very carefully by repeatedly using trial and error technique. While a very large population can slow

down the execution of genetic algorithm, a smaller population may not have ‘good’ mating pool. Therefore, an optimal population size needs to be decided by trial and error.

10.2.7 Population Initialization

In a genetic algorithm, population can be initialized in two ways.

1. **Random initialization** populates random solutions in the initial population. This technique helps to get more optimal solutions.

   The population is usually defined as a two-dimensional array of − size population, size x, chromosome size.
2. **Heuristic initialization** uses a known heuristic for the problem to populate the initial population. Note that heuristic initialization should not be used for the entire population as it may in the population having similar solutions with least diversity. Therefore, we can just seed the population with a few good solutions and fill up the rest with random solutions.

10.2.8 Population Models

Genetic algorithms widely use two types of population models:

Steady state, also known as **incremental GA**, produces one or two offsprings in each iteration. These off-springs replace one or two individuals from the population.

Generational genetic algorithms generate ‘n’ off-springs in each iteration. Here, n refers to the size of the population. In this case, the entire population is replaced by the new one after every iteration.

10.2.9 Fitness Function

The fitness function takes a candidate solution to the problem as input and produces a results statement as output. This output describes how ‘fit’ (or how ‘good’) the solution is with respect to the given problem. The fitness function should be fast as it will be repeatedly applied to find the fitness value. Hence, a slow fitness function can adversely affect the per-

formance of a genetic algorithm. A good fitness function must possess the following characteristics.

1. It should be fast to compute.
2. It must quantitatively measure how fit a given solution is.

For some complex problems, it may not be possible to directly apply the fitness function to calculate the fitness value. In such cases, fitness approximation is done.

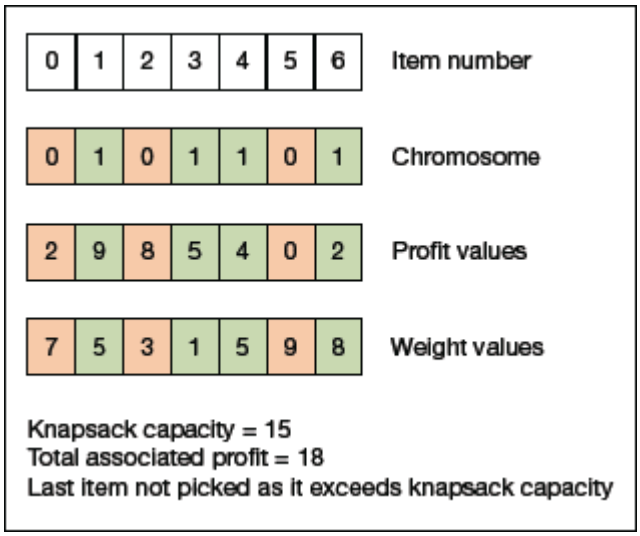


FIGURE 10.12 Calculating Fitness value for a solution of the 0/1 Knapsack problem.

Consider **Fig. 10.12** which demonstrates the calculation of fitness value for a solution of the **0/1 Knapsack problem**. A simple fitness function is applied which adds the profit values of the items that have been picked. Note that these items have a '1' in their chromosome. We start from the leftmost end and continue to move toward the right until the knapsack is full and cannot take more items. Here, starting from left, we will add values for Item Number 1, 3 and 4, giving a score of 9 + 5 + 4 = 18. We cannot include Item Number 6 because the knapsack capacity will be exceeded. Currently, it is obtained by adding weights of Item Number 1, 3 and 4 which is 5 + 1 + 5 = 11. If we add Item Number 6, then the capacity would be 11 + 8 = 19, which is more than 15 and hence, not possible.

10.2.10 Parent Selection

As the name suggests, parent selection refers to the process of selecting individuals from the population (or parents) which will mate and recombine to generate off-springs for the next generation. Parent selection is a very important task as only 'good' parents can produce better and fitter solutions. Therefore, the convergence rate of a genetic algorithm depends on this selection.

While selecting parents, utmost care must be taken to ensure that one extremely fit solution should not take over the entire population in a few generations, as this will diminish diversity in the population and all the solutions produced will be similar to other solutions. One extremely fit solution taking up of the entire population results in a situation known as **premature convergence.** Clearly, it is an undesirable condition in a genetic algorithm.

The most popularly used technique for selecting parents is the fitness proportionate selection. In this technique, every individual can become a parent with a probability which is proportional to its fitness. This means that fitter individuals have a higher chance of mating and propagating their features to the next generation. Fitness proportionate selection puts selection pressure to fitter individuals in the population, generating better individuals over time.

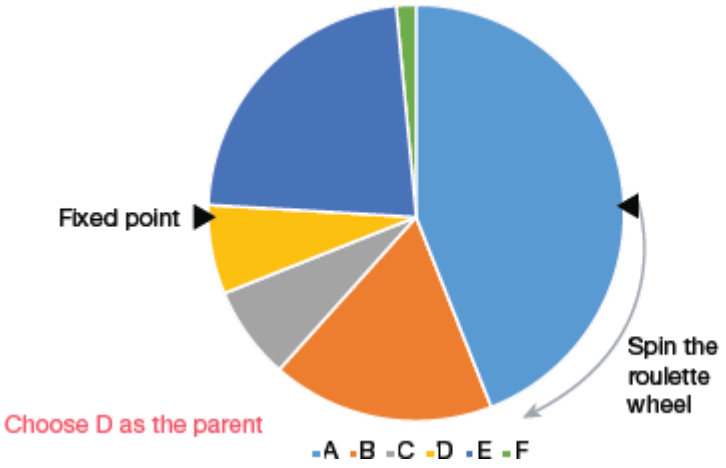
Consider a circular wheel which is divided into **n pies**, where n is the number of individuals in the population. Each individual gets a portion of the circle that is in proportion to its fitness value. Given below are two implementations of this technique.

Roulette Wheel Selection

From **Fig. 10.13**, we see that chromosome A gets a bigger portion of the circle, since it has the highest fitness score. Similarly, other chromosomes have a portion proportional to their fitness score.

In a roulette wheel selection, we have a fixed point on the wheel circumference. When the wheel is rotated, the region of the wheel which comes

in front of the fixed point is chosen as the parent. To find the second parent, the same process is repeated.



Chromosome	Fitness value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

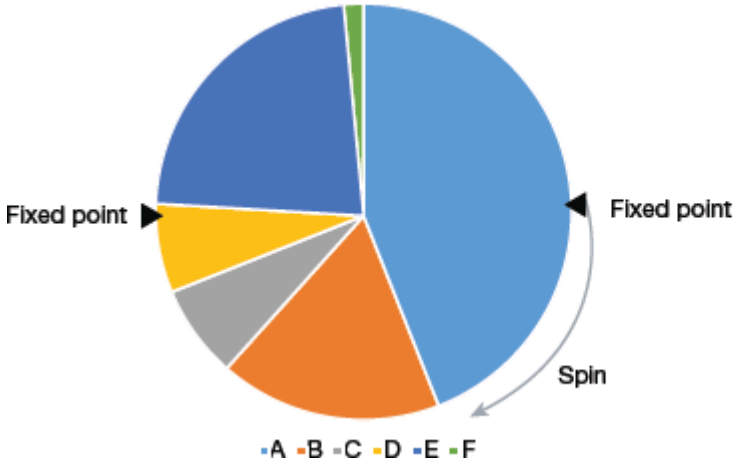
FIGURE 10.13 Roulette Wheel Selection

Since fitter individual has a larger portion of pie on the wheel, it also has a higher chance of coming in front of the fixed point when the wheel is rotated. This clearly indicates that the probability of choosing an individual is directly proportional to its fitness value.

Stochastic Universal Sampling (SUS)

SUS is similar to Roulette wheel selection, but instead of having just one fixed point, the technique has more than one fixed point as shown in **Fig. 10.14**. Therefore, the parents are selected in a single spin of the wheel. Another advantage of SUS is that it increases the probability of selecting the highly fit individuals at least once (refer **Fig. 10.15**).

Fitness proportionate selection methods do not work for cases where the fitness value is negative.



Chromosome	Fitness value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

FIGURE 10.14 Stochastic Universal Sampling

Chromosome	Fitness Value	Rank
A	8.1	1
B	8.0	4
C	8.05	2
D	7.95	6
E	8.02	3
F	7.99	5

FIGURE 10.15

Tournament Selection

In K-Way tournament selection, K individuals are randomly selected from the population and then the best out of K individuals is selected to become a parent (as illustrated in **Fig. 10.16**). The same process is repeated for selecting the other parent. The main reason for wide acceptability of Tournament Selection is that it can even work with negative fitness values.

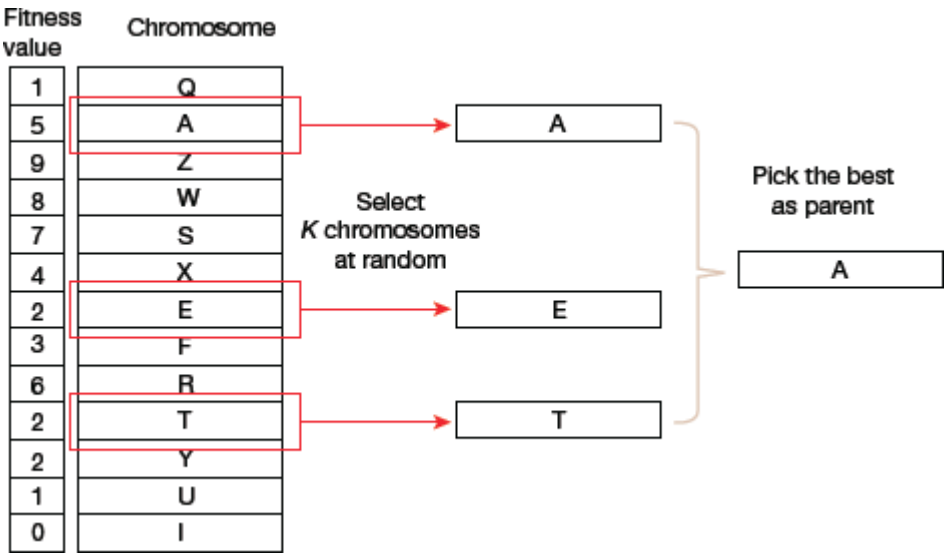
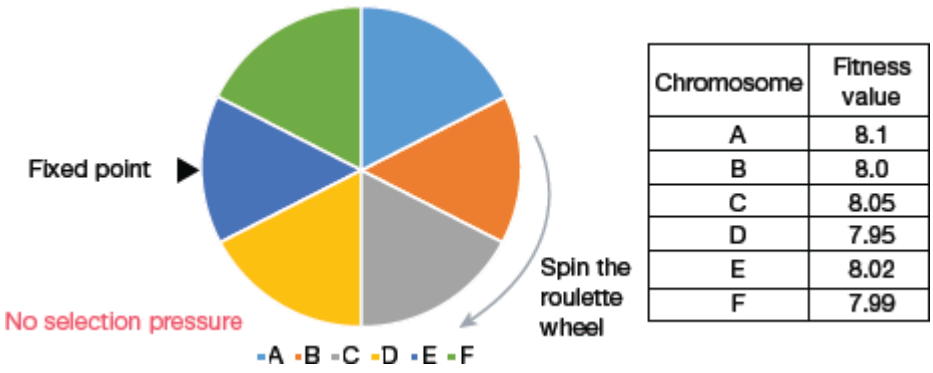


FIGURE 10.16 Tournament Selection

Rank Selection

Rank selection is used when individuals in the population have very close fitness values. In such a situation, if fitness proportionate selection is used, then each individual shares an equal portion of the pie as shown in **Fig. 10.17**. This indicates that irrespective of one's fitness value, every individual has nearly the same probability of being chosen to become a parent. Clearly, in this process, there is a reduced focus on selecting fitter individuals, thereby making the genetic algorithm make poor parent selections and thus gives disappointing results.



Chromosome	Fitness value
A	8.1
B	8.0
C	8.05
D	7.95
E	8.02
F	7.99

FIGURE 10.17 Rank Selection

In such a situation, a better technique is rank the individuals in the population based on their fitness values. Therefore, the selection of the parents depends on the rank of each individual and not its fitness score. Higher

the rank, higher is the probability of being chosen as a parent. Rank selection can work with negative fitness values as well.

Random Selection

In this strategy, parents are selected randomly from the existing population. There is no pressure to select fitter individuals. Because of this randomness, this technique is rarely used.

10.2.11 Crossover

The crossover operator is analogous to reproduction and biological crossover. In this technique, more than one parent is selected and one or more off-springs are produced using the genetic material of the parents. Given below are some crossover operators which are generic in nature. However, designer of the algorithm may even implement a problem-specific operator.

1. **One Point Crossover:** In this technique (illustrated in **Fig. 10.18**), a random crossover point is selected. Then, new off-springs are generated by swapping the tails of its two parents.

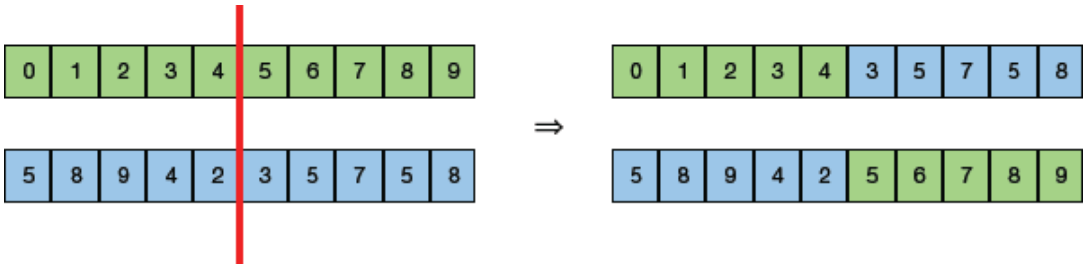


FIGURE 10.18 One Point Crossover

2. **Multi-point Crossover:** It is a generalization of the one-point crossover as it produces new off-springs by swapping alternating segments. Note that in **Fig. 10.19**, we have two (not one) crossover points.

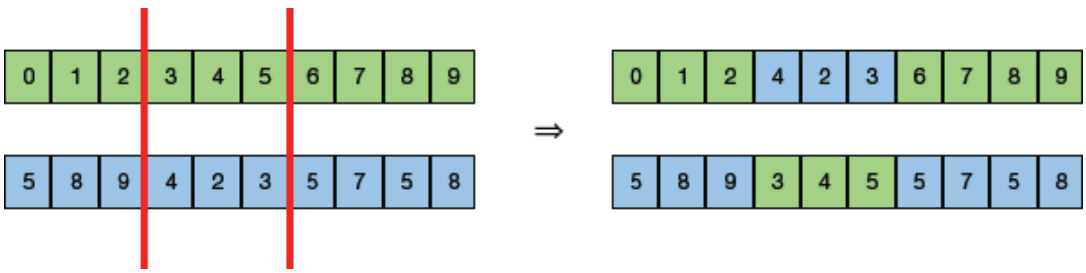


FIGURE 10.19 Multi-point Crossover

3. **Uniform Crossover:** Instead of dividing the chromosome into segments, uniform crossover treats each gene separately (as shown in **Fig. 10.20**). Each gene may or may not be included in the offspring. To determine whether to include that genet or not, a coin is flipped for each chromosome. We can also add a bias in this technique to bias the coin to one parent. This will help the offspring to get more genetic material that particular parent.

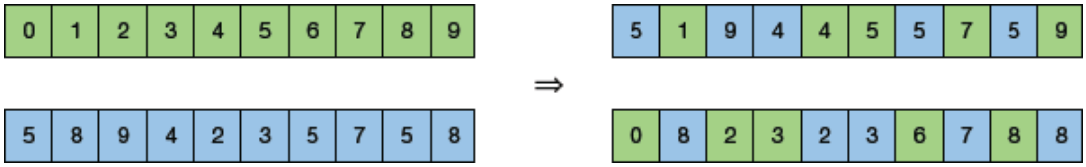


FIGURE 10.20 Uniform Crossover

4. **Whole Arithmetic Recombination:** For integer representations, we take the weighted average of the two parents by using the formula,
   1. Child1 = α.x + (1−α).y
   2. Child2 = α.x + (1−α).y

   So, diverse offspring can be obtained by varying they value of α . If α = 0.5, then both the children will be identical as shown in **Fig. 10.21**.



FIGURE 10.21 Whole Arithmetic Recombination

5. **Davis' Order Crossover (OX1):** OX1 is used for permutation-based crossovers. Follow the steps given below to apply this operator.

Step 1: Create two random crossover points in Parent 1.

Step 2: Copy the segment between the two crossover points from the first parent.

Step 3: Starting from the second crossover point in the second parent, copy the remaining unused numbers from the second parent to the child.

Repeat the steps while wrapping around the list.

Step 4: If you desire a second child from the two parents, flip Parent 1 and Parent 2 and go back to Step 1.

Example: Consider the solution given in **Fig. 10.22**. We have two parents.

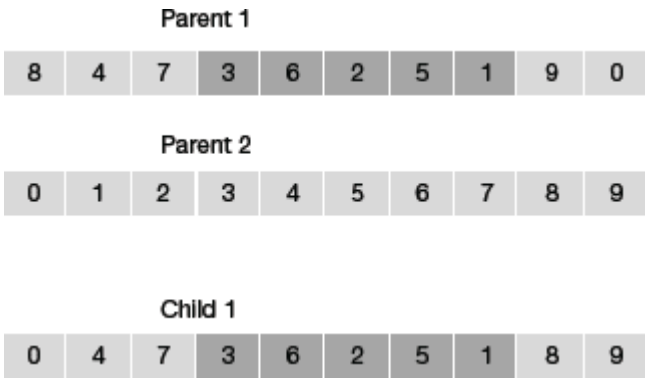


FIGURE 10.22 Davis' Order Crossover

The child is produced by copying elements 3, 6, 2, 5, 1 from the first parent (according to Step 2).

Then, we copy elements from parent 2 that have not been used so far. Starting on the right side of the second crossover point, grab alleles from parent 2 and insert them in Child 1 at the right edge of the second crossover point. Since 8 is in that position in Parent 2, it is inserted into Child 1 first. Similarly, element 9 and 0 from Parent 2 are copied in Child 1 at the right of second crossover point (while wrapping around the list).

Since elements 1 and 2 from parent 2 are already in child, we will not copy them again.

Copy elements 4 and 7 from Parent 1 to complete the chromosome.

10.2.12 Mutation

Mutation is the process in which a new solution is obtained by introducing a small random tweak in the chromosome. Mutation plays an important role in genetic algorithms as it maintains as well as introduces diversity in the genetic population. Mutation performs 'exploration' of the search space and is very essential for a genetic algorithm to converge successfully.

In this section, we will discuss some widely used mutation operators. A genetic algorithm designer can use one of operators or a combination of two or more operators or use a problem-specific mutation operator.

1. **Bit Flip Mutation:** In this technique (illustrated in **Fig. 10.23**), one or more bits are selected randomly and flipped. However, it is applied on binary encoded genetic algorithms.

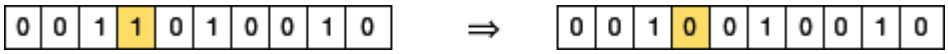


FIGURE 10.23 Bit Flip Mutation

2. **Random Resetting:** It is an extension of the bit flip technique. Random setting operator is used for genetic algorithms that use integer representation. It selects a random value from the set of permissible values and assigns it to a gene that is again chosen randomly.
3. **Swap Mutation:** According to this approach (demonstrated in **Fig. 10.24**), two positions on the chromosome are selected randomly and their values are swapped (interchanged). This operator is commonly used in permutation-based encodings.

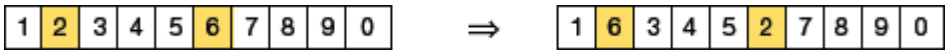


FIGURE 10.24 Swap Mutation

4. **Scramble Mutation:** It is a popular technique used with genetic algorithms that use permutation representations. As the name suggests, scramble mutation selects a subset of genes from the chromosome and then randomly scrambles (or shuffles) their values as shown in **Fig. 10.25**.

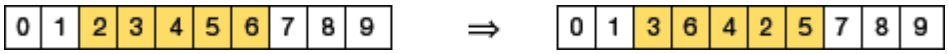


FIGURE 10.25 Scramble Mutation

5. **Inversion Mutation:** As in scramble mutation (refer **Fig. 10.26**), inversion mutation also choses a subset of genes but instead of shuffling the subset, the string in the entire subset is inverted.

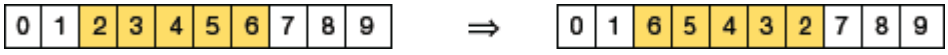


FIGURE 10.26 Inversion Mutation

Survivor Selection

The survivor selection policy is used to determine which individuals will be a part of the next generation and which will be discarded. This is an important step to ensure that the fitter individuals are passed on to the next generation. We also need to ensure that diversity in the population should be maintained in every generation.

Some genetic algorithms use the strategy of **elitism,** which ensures that the fittest member of the current population is always propagated to the next generation. Therefore, under no circumstance can the fittest member of the current population be replaced. Though it is the simplest way to kick random members out of the population, this approach frequently suffers from convergence issues. Therefore, policies discussed below are widely used.

Age-Based Selection

Age-based selection does not consider fitness value for selection. It allows each individual in the population to reproduce for a finite generation. After that, irrespective of how good it is, it is kicked out of the population.

For example, in **Fig. 10.27**, we see that age is a number that specifies for how many generations the individual has been in the population. The oldest members of the population, P4 and P7 are kicked out of the population and the ages of the other members are incremented by one.

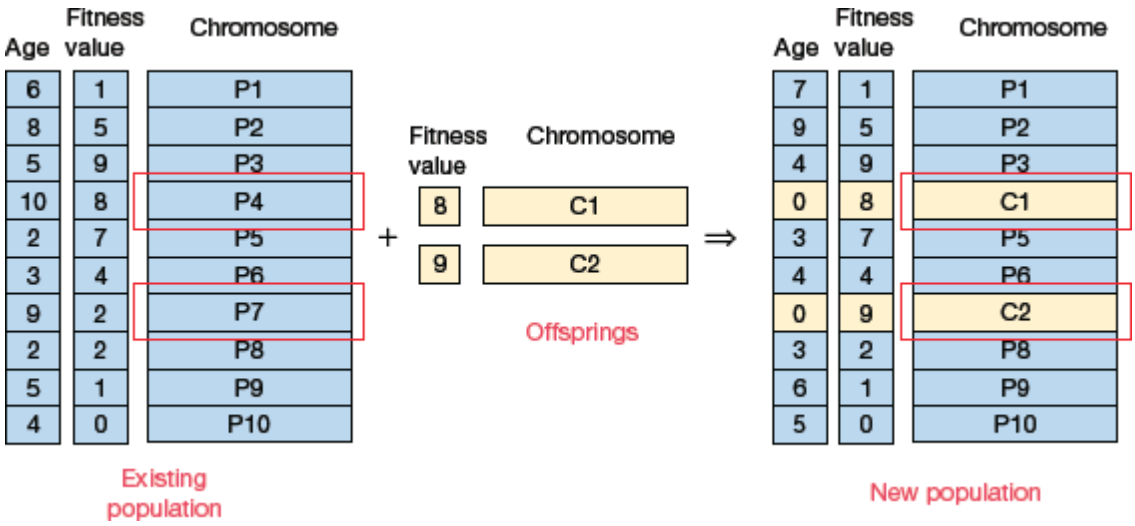


FIGURE 10.27 Age-Based Selection

Fitness-Based Selection

In this selection strategy, the children replace the least fit individuals in the population. The least fit individuals can be selected using any selection policies (like tournament selection, fitness proportionate selection, etc.).

For example, in **Fig. 10.28**, we see that the least fit individuals P1 and P10 of the population are replaced by the children. Note that P1 and P9 also have the same fitness value, so even they could have been removed. The decision of which individual to remove from the population is arbitrary.

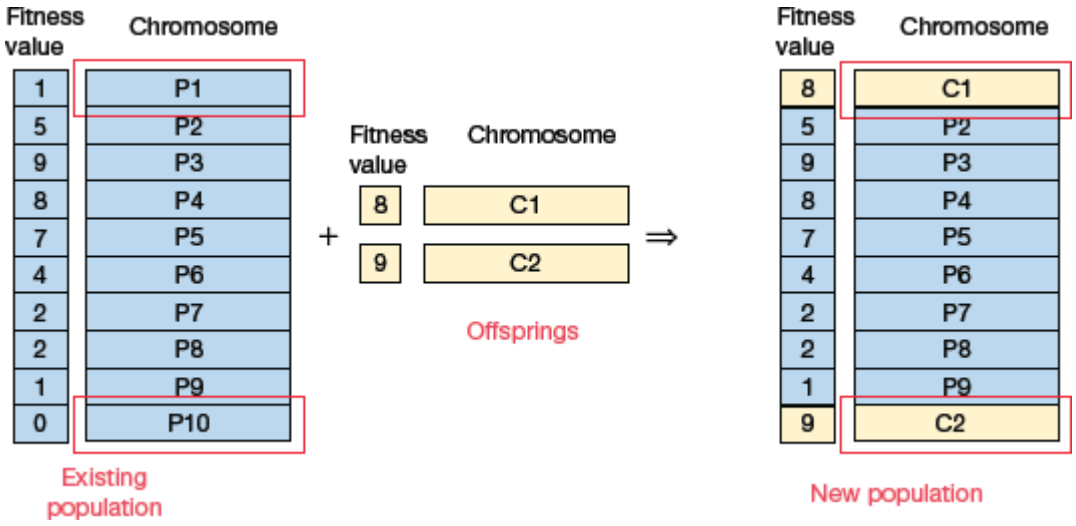


FIGURE 10.28 Fitness-Based Selection

10.2.13 Termination Condition

The termination condition of a genetic algorithm determines when the execution of the algorithm will halt. It has been observed that a genetic algorithm initially works very fast and gives good better solutions in subsequent iterations. But once the saturation point is reached, the improvements in solutions become almost negligible. At such a point, it is better to terminate the algorithm. The termination condition must give a solution that is close to the optimal. The algorithm can terminate under the following conditions:

1. When there is no improvement for X iterations in the population.
2. When the absolute number of generations has been reached.
3. When a certain pre-defined value of the objective function value has been reached.

To implement termination condition in a genetic algorithm, we can maintain a counter (with initial value = 0), to track the generations for which there has been no improvement in the population. When the generated off-spring is not better than the individuals in the population, the counter is incremented.

If the fitness value of any of the off-springs is better than other individuals in the population, then the counter is again reset to zero. The algorithm terminates when the value of the counter becomes equal to a predetermined value. Like other parameters, the termination condition of a genetic algorithm is specific to the problem at hand. And the designer of the genetic algorithm should try various options to see what best-suits that particular problem.

10.2.14 Application Areas

Genetic algorithms are widely used in a variety of application areas as given below.

1. ***Optimization:*** Genetic algorithms are widely used in optimization problems. The goal here is to either maximize or minimize the value of a given objective function as per the specified constraints. Genetic

algorithms can also be used for multimodal optimization to find multiple optimum solutions. Travelling salesman problem (TSP) can also solve optimally using genetic algorithms.

2. ***Economics:*** Economic models (like the cobweb model, game theory equilibrium resolution, asset pricing, etc.) are characterized using genetic algorithms.
3. ***Neural Networks:*** Genetic algorithms are used to train neural networks, particularly recurrent neural networks.
4. ***Parallelization:*** Genetic algorithms can perform parallel computations. Therefore, these algorithms can effectively solve certain complex problems.
5. ***Image Processing:*** Digital image processing (DIP) and dense pixel matching can be easily performed using genetic algorithms.
6. ***Scheduling applications:*** Scheduling problems, particularly the time tabling problem, can easily be solved by genetic algorithms.
7. ***Machine Learning:*** Genetic algorithms is an important concept used in machine learning algorithms.
8. ***Robot Trajectory Generation:*** Genetic algorithms can be used to plan the path which will be taken by a robot arm while moving from one point to another.
9. ***Parametric Design of Aircraft:*** Better aircraft designs can be obtained by applying genetic algorithms while varying certain important parameters.
10. ***DNA Analysis:*** Genetic algorithms when applied on spectrometric data about the sample, can be used to determine the structure of DNA.

10.3 Soft Computing

Soft computing refers to a group of computational techniques that are based on artificial intelligence (AI) and natural selection. They are often used to develop cost-effective solutions for complex real-world problems that are either difficult or impossible to be solved by conventional computing techniques.

Soft computing differs from conventional computing as it deals with approximation models.

The term 'soft computing' was first coined in 1992 by **Zadeh** to provide precise approximation and quick solutions for complex real-life problems. For example, a soft computing system can be used for character recognition as shown in **Fig. 10.29**.



FIGURE 10.29 Soft Computing

Soft computing is an emerging domain that can map a human mind. In fact, the human mind is a role model for soft computing.

Computation is a process of converting input of one form to another form using certain control actions. The input is called an antecedent and the output is called the consequent.

10.3.1 Characteristics of Soft Computing

Soft computing provides an approximate but precise solution for real-world complex problems that cannot be solved by conventional computing techniques.

Soft computing algorithms are not affected by changes in the environment. This makes these algorithms adaptive for the current process.

Soft computing algorithms do not require any mathematical model to solve the problem as it learns **from experimental data.**

Solutions given by soft computing are uncertain since we get different solutions on solving the same problem using the same set of input data at different intervals of time. Refer **Fig. 10.30** to understand how soft computing differs from hard computing

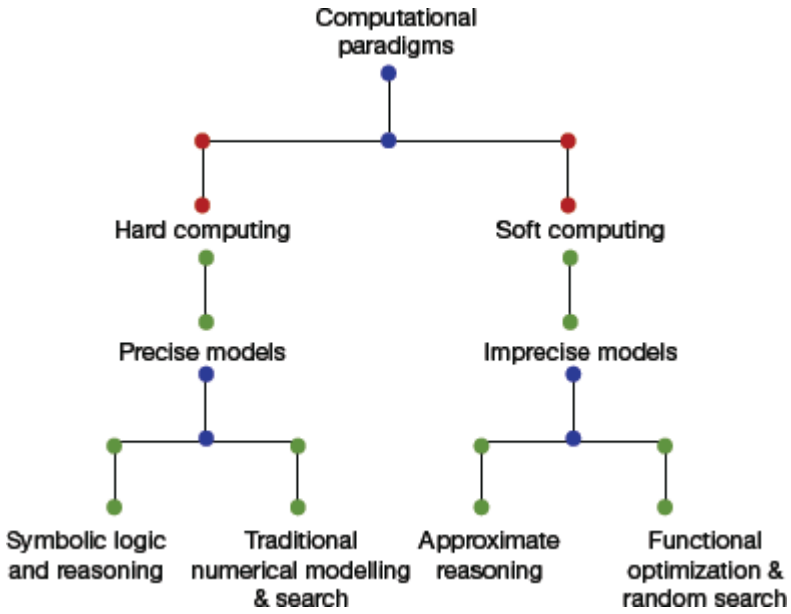


FIGURE 10.30 Computational paradigms and Soft Computing

It is based on Fuzzy logic, genetic algorithms, machine learning, ANN and expert systems. **Table 10.4** draws comparison between hard and soft computing.

- Soft-computing algorithms are robust as they can work well even in the presence of noise in the input data.
- Soft computing provides low-cost solutions for problems which could be computationally very expensive if solved using hard computing.

10.3.2 Need for Soft Computing

1. Hard computing or conventional techniques are usually good for solving mathematical problems that need a precise answer. But when the task is to solve a complex real-world problem where either approximate solution works better or there exist no precise solution, then soft computing is the only possibility.
2. For applications that need to map human mind, soft computing is the only viable technique. For better understanding of the concept, refer **Table 10.4** and **Table 10.5** that makes a comparison between Soft Computing and Hard Computing and between Soft Computing and AI respectively.

TABLE 10.4 Soft computing vs hard computing

Parameters	Soft Computing	Hard Computing
Uses	Uses techniques that mimics the way human brain works	Mathematical concept to solve a problem
Output	Approximate answer	Precise answer
Application	Complex real-world problems	Any numerical problem
Computation time	Takes less computation time.	Takes more computation time.
Dependency	It depends on approximation and disposition.	It is mainly based on binary logic and numerical systems.
Computation type	Parallel computation	Sequential computation
Techniques	Neural networks, Fuzzy logic, genetic algorithms, etc.	Any numerical or traditional methods of solving using personal computers.
Used for	Handwriting recognition, image classification, music generation, etc.	Finding derivatives, integrals, searching and sorting algorithms, finding the shortest path between two points and many more

TABLE 10.5 Differences between AI and Soft Computing

Artificial Intelligence	Soft Computing
AI is the art and science of developing intelligent machines.	Soft computing aims to exploit tolerance for uncertainty, imprecision and partial truth.
It is mainly used to find missing pieces between the interesting real-world problems.	They are used to handle imprecision, uncertainty and partial truth.
Branches of AI are reasoning, perception and natural language processing.	Branches of soft computing are Fuzzy systems, evolutionary computation and artificial neural computing.
The goal is to stimulate human-level intelligence in machines.	The goal is to work with the pervasive imprecision of the real world.
They require programs to be written.	They do not depend on pre-written programs. Rather, they can evolve their own programs.
They require exact input sample.	They can work with ambiguous and noisy data.

Example: Let us consider a problem that cannot be solved by traditional computing, but soft computing gives an approximate solution. If you ask, 'is the tea is hot or cold?' Traditional computing techniques may give an answer to this question but when asked, 'how much hot?', they will not be able to tell. Soft computing rescues this situation by saying, 'moderately hot'.

Advantages

1. Applies simple mathematical calculations
2. Gives efficient solutions
3. Can be applied to solve problems in real-time
4. Based on human reasoning

Disadvantages

1. Instead of precise, it gives approximate results.
2. Even a small error may halt the entire system. In such a case, the system must be corrected from the beginning. And this may take more time to process input data to give the output.

10.3.3 Applications of Soft Computing

1. Soft computing can be used to control motors automatically like the induction motor.
2. Intelligent control systems to control power plants can be designed using soft computing.
3. Soft computing takes an image, music or video as input to get an exact duplicate of the original image, music or video.
4. In biomedical field, soft computing techniques (refer **Fig. 10.31**) are used for diagnosis, monitoring, treatment and therapy.
5. To perform a task, smart devices automatically communicate with other devices using different communication protocols. There is no proper standard protocol to communicate and this often creates problem. Soft computing techniques are used in such a situation to facilitate communication between smart devices with high privacy and robustness.

In fact, there are many more applications where soft computing is widely used.

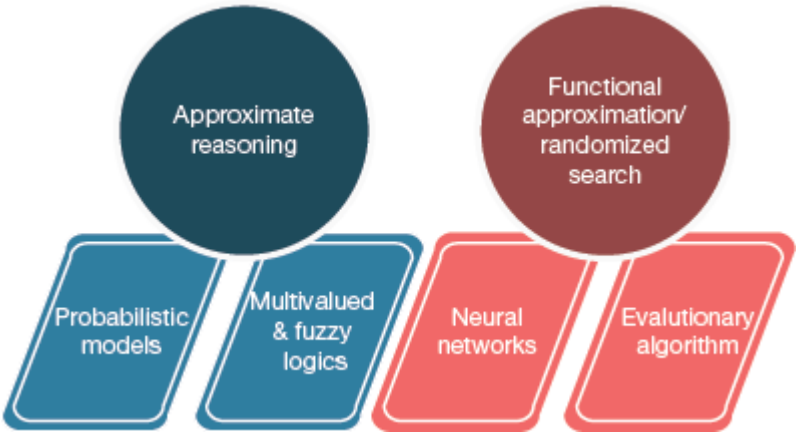


FIGURE 10.31 Soft Computing Techniques

10.3.4 Elements of Soft Computing

Soft computing lays the foundation for an emerging field of conceptual intelligence. Technologies like fuzzy logic (FL), machine learning (ML), neural network (NN), probabilistic reasoning (PR) and evolutionary computation (EC) are the supplements of soft computing and are used to resolve any complex problem.

Fuzzy Logic

As discussed, fuzzy logic applies mathematical concepts to solve problems with an open and imprecise spectrum of data to get an array of precise conclusions.

Fuzzy logic achieves the best possible solution to complex problems using the available information on the input data. Fuzzy logic is considered the best solution finder.

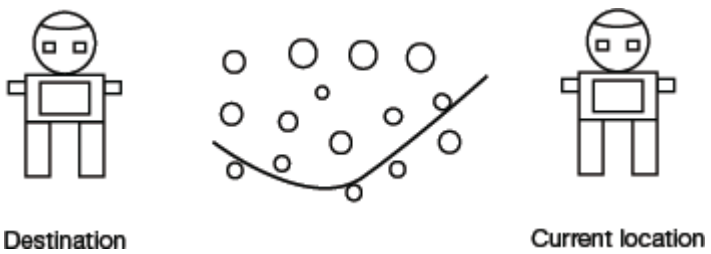


FIGURE 10.32 Fuzzy Logic for movement of a Robot

Example: Fuzzy logic is used to control the movement of a robot (refer **Fig. 10.32**) from one place to another within a short time where there are

many obstacles on the way. To calculate its movement to reach the destination without colliding to any obstacle, fuzzy solutions may give better results than precise solutions.

Probabilistic Reasoning

The technique applies the concept of probability to determine the uncertainty of a given event. The probability of an event is calculated using the below formula:

$$\textit{Probability of occurrence} = \textit{number of desired outcomes} \;/\; \textit{total number of outcomes}$$

Probabilistic reasoning is mainly used in the following scenarios:

1. When conducting an experiment, we get an unknown error.
2. When we are not sure about uncertain outcomes.
3. When number of predicates are too large to handle.

Neural Network (ANN)

Neural networks, developed in the 1950s, facilitate users to solve real-world problems, which a traditional program can never solve. They work in the same way as the human brain, by emulating a network of neurons that makes a human brain. ANN enables a machine to mimic the human brain so that it can think and take an effective decision. The output obtained is mathematically justifiable on the current input data based on past information stored.

A neural network is made of several interconnected layers. Each layer has one or more nodes for processing inputs received. A layer accepts the output of the previous layer as its input, processes it to give the output to the layer succeeding it. This is known as forward propagation.

After giving the final output, backward propagation starts. In this phase, the result obtained from neural network is compared with the actual output. If an error is identified (due to forward propagation), then the task is to propagate backwards in the network to minimize the value/weight of that neuron(s) that are responsible for the error.

Soft computing usually uses deep neural networks. A deep neural network has a greater number of hidden layers as compared to a simple ANN. While a traditional neural network has two to three hidden layers, a deep neural network, on the other hand, has as many as 150 layers.

Genetic Algorithms (GA)

Genetic algorithm finds solutions by deriving concepts from natural selection and survival of the fittest which are very important concepts in genetics.

Genetic algorithms that come under the category of evolutionary algorithms were introduced by Prof. John Holland in 1965. They mainly solve optimization problems like maximization and minimization of objective functions.

Genetic algorithms are based on heuristic or randomized search that provides an initial set of solutions and then generates a solution to the NP-hard problems efficiently and effectively in real time. This means that complex problems which cannot be solved mathematically can be easily solved by using genetic algorithm.

Example: A person wants to invest some money in the bank. There are numerous banks where he can invest in different schemes and policies (refer **Fig. 10.33**). The decision regarding bank and policy for investment must be taken to maximize profits. In such a scenario, application of evolution computing using genetic algorithms gives the best results.

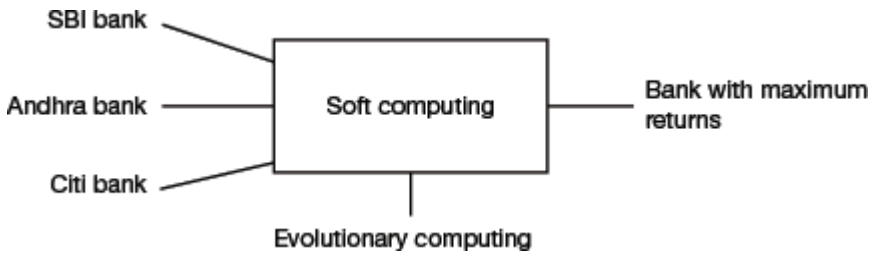


FIGURE 10.33 Genetic Algorithm for Selecting Bank and its Policies

When applying genetic algorithm, remember that the initial population contains several solutions to a concerning problem. Let us quickly enu-

merate the steps performed to solve a problem using genetic algorithm.

Step 1: The algorithm calculates the fitness of each individual solution and provides them with the fitness score. Higher the fitness score, higher is the probability of a solution being selected for the reproduction process.

Step 2: The algorithm selects the solution with the highest fitness score.

Step 3: Perform crossover on the selected individual from the population to get an offspring.

Step 4: Add this offspring to the initial population.

Step 5: To maintain population diversity, mutation phase inserts random genes into the offspring.

The algorithm will stop producing the offsprings when they are different than the previous generation.

10.4 Transfer Learning

Transfer learning is the reuse of a pre-trained model on a new problem (as shown in **Fig. 10.34**). So, a machine exploits the knowledge gained from a previous task to improve generalization about another. For example, a classifier that is trained to predict whether an image contains food or not can also be used to recognize drinks with the help of knowledge it gained during the training process.

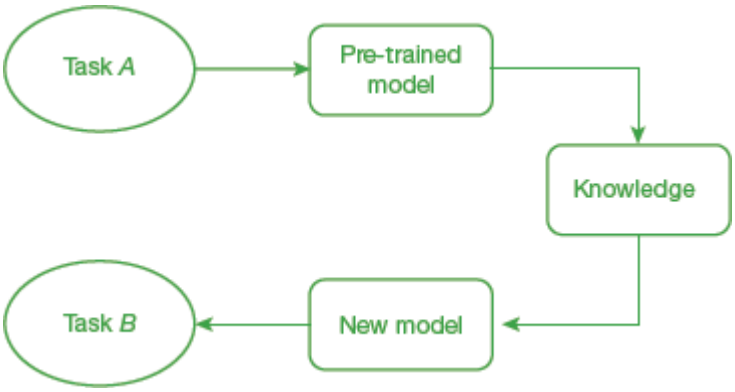


FIGURE 10.34 Transfer Learning

It is currently being used in deep learning in which deep neural networks are trained with very limited data. Since most of the real-world applications do not have millions of labelled data points to train complex models, transfer learning is an important concept.

In transfer learning, we transfer the weights that a network has learned at 'task A' to a new 'task B.' While task A has a lot of labelled training data available, task B on the other hand, lacks volumes of data. In such a scenario, transfer learning starts the learning process with patterns learned from solving a related task rather than starting from the scratch.

Though transfer learning is not a machine learning technique, it is popularly used in combination with neural networks that require huge amounts of data and computational power. Applications like computer vision and natural language processing extensively use this technique.

10.4.1 How Transfer Learning Works?

In a computer vision application, neural networks detect edges in the earlier layers, shapes in the middle layer and some task-specific features in the later layers. Similarly, in transfer learning, we use the early and middle layers of the model and retrain the latter layers. It helps to take advantage of labelled data of the task it was initially trained on and then transfer as much knowledge as possible from the previous task.

10.4.2 When to Use Transfer Learning

Some guidelines which dictate when transfer learning might be used are as follows and summarized in **Fig. 10.35**:

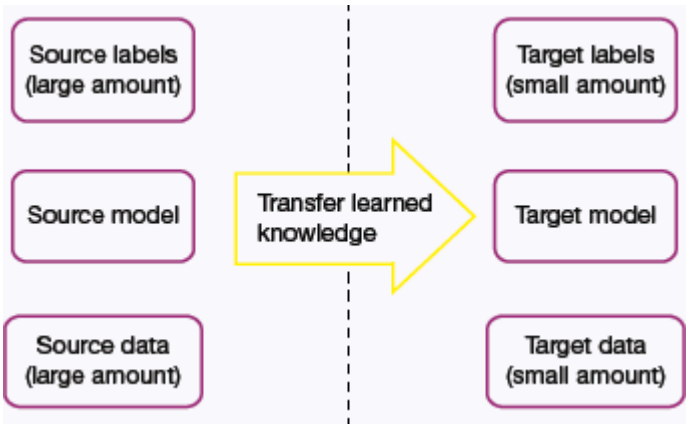


FIGURE 10.35 When to Use Transfer Learning

1. When there is very limited labelled training data available to train the neural network from scratch.
2. When there is a neural network that has already been trained on a similar task with massive amounts of data.
3. When the two tasks have the same input.
4. If the existing model was trained using an open-source library (like TensorFlow), you can simply restore some layers and retrain others for the task at hand.

Transfer learning is an optimization, a shortcut to save time and get better performance. Lisa Torrey and Jude Shavlik described three possible benefits to look for when using transfer learning:

1. **Higher start**: The initial skill on the source model before even the model was refined, is higher than it otherwise would be (refer **Fig. 10.36**).
2. **Higher slope**: The rate of improvement of skill during training of the source model is steeper than it otherwise would be.
3. **Higher asymptote**: The converged skill of the trained model is better than it otherwise would be.

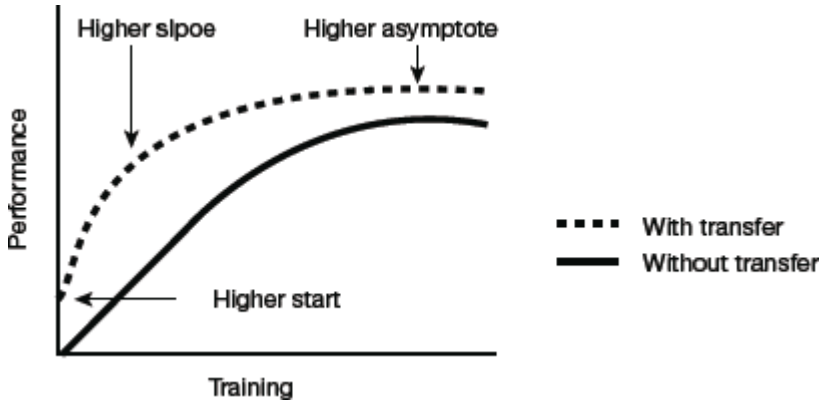


FIGURE 10.36 Graph depicting merits of Transfer Learning

However, the choice of source data or source model is an open problem and may require domain expertise and/or Three ways in which transfer might improve learning intuition developed via experience.

Note that transfer learning only works in the following situations:

1. If the features learned from the first task are general so that they can be useful for another related task.
2. Input of the model needs to have the same size as it was initially trained with. In case this condition is not satisfied, then resize the input to the needed size.

You can visit pretrained. ml which offers a list of sortable and searchable compilation of pretrained deep learning models with demos and code.

10.4.3 Approaches to Transfer Learning

Training a Model to Reuse It

To solve a task A having limited data to train a deep neural network, find a related task B with an abundance of data. Train the deep neural network on task B and use the same model to solve task A. Whether to use the entire model or just few layers of the model will depend on the problem at hand.

If the input for both the tasks is same, then re-use the model, else make some changes in the task-specific layers of the model before using it.

Using a Pre-Trained Model

Use a pre-trained model but do some research to choose the best one from all available options. Again, whether to reuse the entire model or re-train few task-specific layers depends on the problem to solve. For example, Keras provides numerous pre-trained models that can be used for transfer learning, prediction, feature extraction and fine-tuning. This type of transfer learning is most commonly used throughout deep learning.

Some popular pre-trained machine learning models include the Inception-v3 model, which was trained for the ImageNet to classify images into 1,000 classes like ‘zebra,’ ‘Dalmatian’ and ‘dishwasher.’

Microsoft also offers some pre-trained models that can be used with both R and Python. Developers just have to incorporate the MicrosoftML R and the Microsoftml Python package. Other commonly used models are ResNet and AlexNet.

The more you want to inherit features from a pre-trained model, the more you have to freeze layers.

Feature Extraction

Use deep learning techniques to discover the best representation of the problem. In other words, identify the most important features. This approach is therefore, known as representation learning, and often gives much better performance than hand-designed representation.

Pre-trained model of ImageNet cannot be used with biomedical images because ImageNet does not contain images belonging to the biomedical field.

The learned representation can then be used to solve other problems. But to solve other problems, only first few layers are reused as they are the ones that identify the most useful features of the problem. Latter layers are retrained as they are task-specific. Feature extraction is extensively used in computer vision as it reduces the size of the dataset (refer **Fig. 10.37**). This, in turn, decreases computation time and enhances the performance of the algorithm.

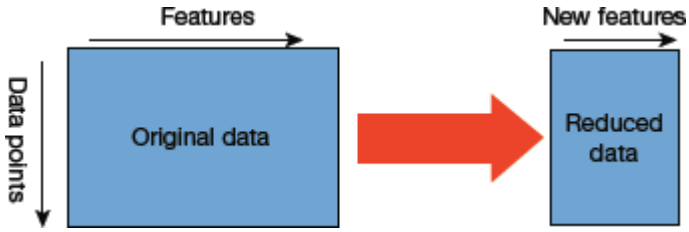


FIGURE 10.37

10.4.4 Traditional Machine Learning vs Transfer Learning

Deep learning experts introduced transfer learning to overcome the limitations of traditional machine learning models. Refer **Table 10.6** and Figure 10.38 to see a comparison between Traditional Machine Learning and Transfer Learning.

TABLE 10.6 Differences between traditional machine learning and transfer learning

Traditional Machine Learning	Transfer Learning
Models require training from scratch.	Models use knowledge from previously learned tasks.
It is computationally expensive.	It is computationally efficient.
It requires a large amount of data to achieve high performance.	It achieves better results using a small data set.
It is an isolated training approach where each model is independently trained for a specific purpose, without any dependency on past knowledge.	It uses knowledge acquired from the pre-trained model to proceed with the task.

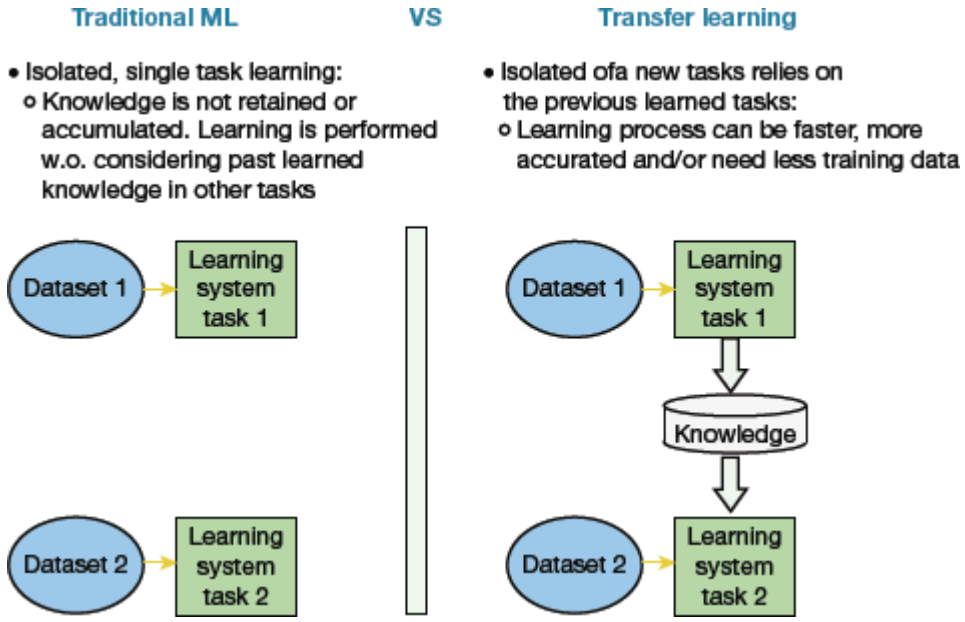


FIGURE 10.38 Traditional Machine Learning vs Transfer Learning

10.4.5 Classical Transfer Learning Strategies

Based on the domain of the application, the problem to solve and the availability of data, we may choose different transfer learning strategies. However, before deciding on the strategy, it is important to have answers for the following questions to improve performance of the target task:

1. *Which* part of the knowledge can be transferred from the source to the targets?
2. *When* to transfer knowledge and when not to?
3. *How* to transfer the knowledge gained from the source model?

Traditionally, there are three types of transfer learning strategies that can be chosen based upon the domain of the task and the amount of labelled/unlabelled data available.

Inductive Transfer Learning

This requires the source and target domains to be the same even if the specific tasks on which the model is working on are different (refer **Fig. 10.39**).

The algorithms try their best to use knowledge from the pre-trained source model and apply them to improve the target task. Since the source model has the expertise on the features of the domain, it is better to start with it than to start from scratch.

Inductive transfer learning is further divided into two groups—multi-task learning and self-taught learning. While task learning is applicable when the source domain contains labelled data, the latter is used when labelled data is not available.

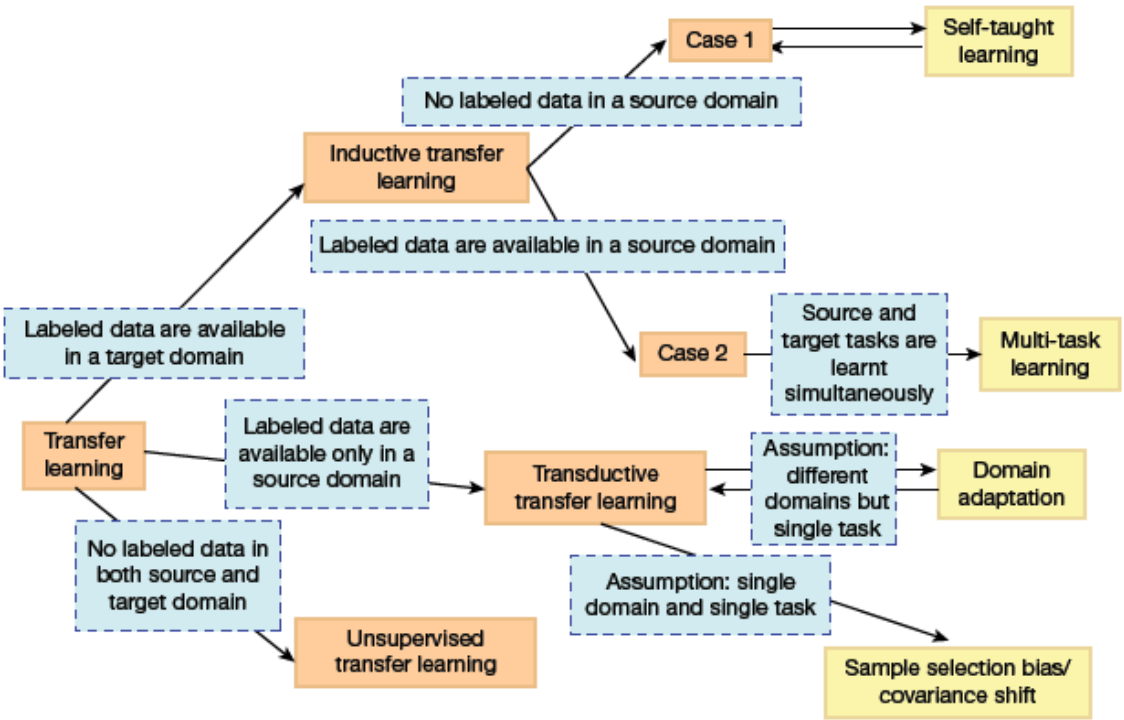


FIGURE 10.39 Inductive Transfer Learning

Transductive Transfer Learning

This type of learning is used when the domains of the source and target tasks are not exactly the same but are interrelated. In this scenario, the source domain has a lot of labelled data but the target domain has only unlabelled data. So, this learning can be used to derive similarities between the source and target tasks.

Unsupervised Transfer Learning

Though this is similar to inductive transfer learning, the algorithms in this approach focus on unsupervised tasks and work on unlabelled datasets both in the source and target tasks.

Transfer learning strategies can also be categorized based on similarity of the domain. This type of division is independent of the type of data samples present for training.

Homogeneous Transfer Learning

As the name suggests, this approach is used when the domains are of the same feature space. There exists only a slight difference in marginal distributions in the domains which can be catered by correcting the sample selection bias or covariate shift.

Instance Transfer

It is done when there is a large amount of labelled data in the source domain, a limited number in the target domain and marginal distributions in both the domains and feature spaces.

For example, imagine that you have to build a model to diagnose cancer in a specific region where there are mostly elderly people. But you have limited target-domain instances. The relevant data are available from another region where the majority comprises young people. In this scenario, directly transferring all the data from another region may not be a viable solution due to the existence of marginal distribution and the fact that elderly have a higher risk of cancer than younger people.

So, here, the marginal distributions are adapted by reassigning weights to the source domain instances in the loss function.

Parameter Transfer

In this technique, it is believed that a well-trained model on the source domain has learned a well-defined structure, and if two tasks are related, this structure can be transferred to the target model. Parameter transfer learning technique transfers the knowledge at the model/parameter level. Knowledge is transferred through shared parameters of the source and target domain learner models. Generally, there are two ways to share the weights-soft weight sharing and hard weight sharing. In ***soft weight sharing***, the model is expected to be close to the already learned features and is penalized if its weights deviate significantly from a given set of weights. In ***hard weight sharing***, the exact weights among different models are shared.

Feature-Representation Transfer

As implicit from the name, this approach transforms the original features to create a new feature representation. This technique can further be categorized as asymmetric and symmetric feature-based transfer learning.

1. *Asymmetric approaches* transform the source features to match the target ones. Therefore, features from the source domain are transformed to make them fit into the target feature space. However, there can be some information loss in this process due to the marginal difference in the feature distribution.
2. *Symmetric approaches* identify a common feature space and then transform both the source and the target features into this new feature representation.

Relational-Knowledge Transfer

This technique focuses on learning the relations between the source and a target domain to extract past knowledge so that it can be used in the

current context. Logical relationship or rules learned in the source domain are then transferred to the target domain.

For example, if we learn the relationship between different elements of the speech in a male voice, it can be then used to analyse the sentence in another voice.

Heterogeneous Transfer Learning

In transfer learning, representations from a previous network are derived to extract meaningful features from new samples for an inter-related task. Though this technique seems simple and straightforward, its applicability is not a trivial task due to the following reasons.

1. There may exist a difference between the source and the target domain and feature spaces.
2. Limited availability of labelled data in the source domain with the same feature space as the target domain.

In such a scenario, heterogeneous transfer learning methods are preferred. This learning strategy is, therefore, applied in cross-domain tasks like cross-language text categorization, text-to-image classification and many others.

10.4.6 Transfer Learning for Deep Learning

Transfer learning is popularly used for applications like NLP and image recognition. There are several pre-trained models that achieve state-of-the-art performance in this field.

Off-The-Shelf Pre-Trained Models as Feature Extractors

We know that deep learning systems comprises multiple layers and each layer learns different features. Initial layers extract higher-level features that narrow down to fine-grained features as we go deeper into the network. These layers are finally connected to the last layer that gives the final output. This opens the scope of using popular pre-trained networks (such as Oxford VGG Model, Google Inception Model, Microsoft ResNet Model) without its final layer to extract a fixed feature for other tasks.

Transfer Learning with Pre-Trained Deep Learning Models as Feature Extractors

In this technique, the pre-trained model's weighted layers are just used to extract features. The model's weights are not updated during training with new data for the new task. Since the pre-trained models are trained on a large and general enough dataset, it effectively serves as a generic model of the visual world.

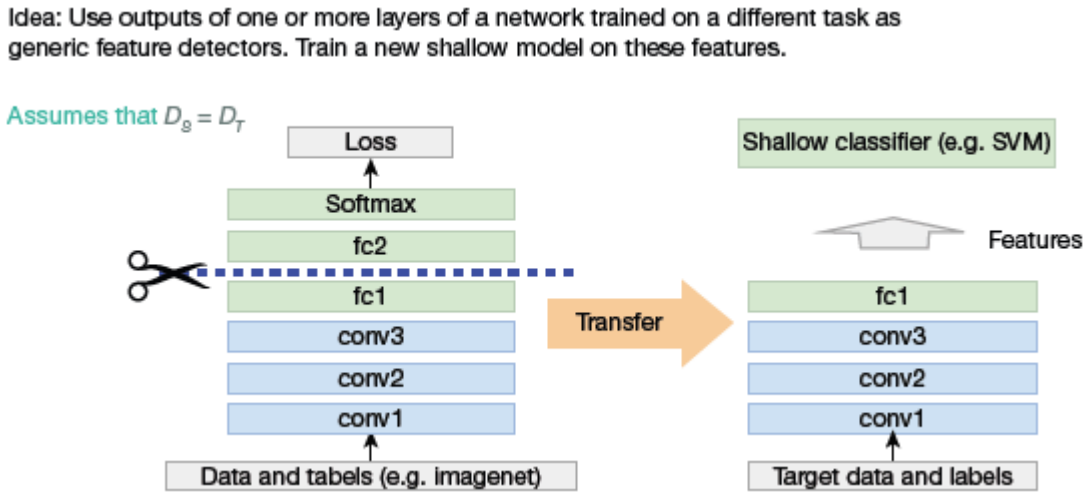


FIGURE 10.40 Transfer learning with Pre-trained deep learning models as feature extractors

Fine Tuning Off-The-Shelf Pre-Trained Models

In this technique, we do not just directly depend on the features extracted from the pre-trained models and replace the final layer. Rather, we selectively retrain some of the previous layers.

Deep neural networks have a layered architecture and have many tunable hyperparameters. While the initial layers capture generic features, the later ones focus on the explicit task at hand. Thus, the higher-order feature representations in the base model must be fine-tuned to make them more relevant for the specific task. We can re-train some layers of the model and keep others frozen in training.

An example is depicted in **Fig. 10.41** on an object detection task, where initial lower layers of the network learn very generic features and the higher layers learn very task-specific features.

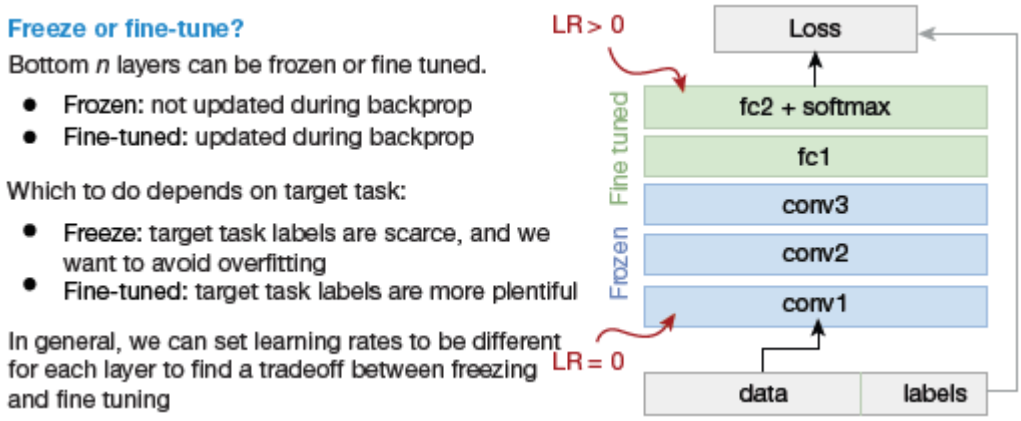


FIGURE 10.41 Freeze vs Fine-tune in Transfer Learning for Deep Learning

Freezing and Fine-Tuning Layers

To increase the model's performance, we often re-train (or 'fine-tune') the weights of the top layers of the pre-trained model and train the newly added classifier. Fine tuning a small number of top layers that are generic rather than the entire model, therefore, it updates the weights for a specific learning and allows the model to apply past knowledge in the target domain and re-learn some things again.

Freezing initial layers to reuse the basic knowledge derived from the past training and fine-tuning higher layers to adapt specialized features to work with the new dataset offers an efficient solution.

10.4.7 Steps in Transfer Learning

Transfer learning can be performed in just a few steps that are illustrated in **Fig. 10.42**.

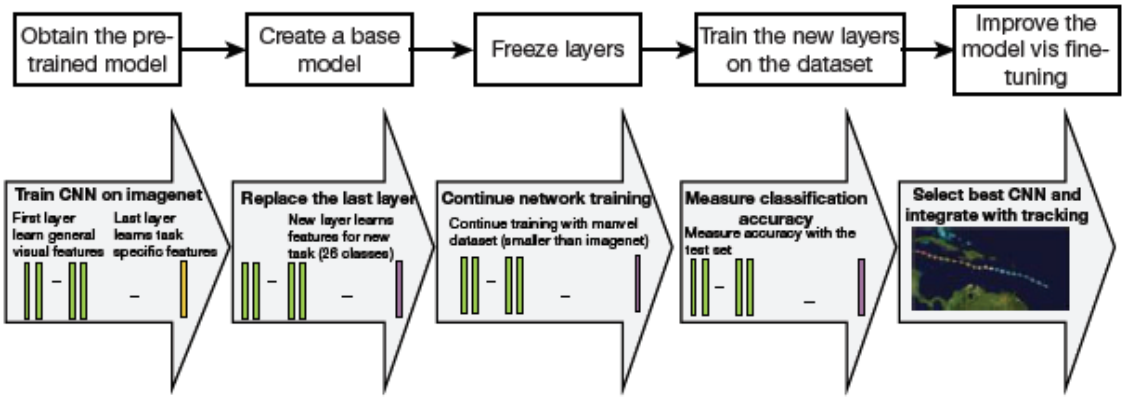


FIGURE 10.42 Transfer Learning process

Step 1: Obtain pre-trained model. The model is kept as the base of our training. Its selection entirely depends on the task at hand. Transfer learning that the knowledge of the pre-trained source model must be directly related and compatible with the target task domain. Few examples of pre-trained models are,

For computer vision: VGG-16, VGG-19, Inception V3, XCeption, ResNet-50

For NLP tasks: Word2Vec, GloVe, FastText

Step 2. Create a base model. The base model is one of the architectures (such as ResNet or Xception) which is in close relation to our task. We can either download the network weights to save time or use the network architecture to train our model from scratch. If the base model has more neurons in the final output layer than required, then we need to remove the final output layer and change it accordingly.

Step 3: Freeze layers. Freeze the starting layers from the pre-trained model to save time in making the model learn the basic features. If we do not freeze the initial layers, the learning already been done will be lost. The process will then be equivalent to training the model from scratch, thereby losing time and other resources.

Step 4: Add new trainable layers. From the pre-trained model, we only use knowledge of feature extraction layers.

Rest of the layers that would perform the specialized tasks needs to be added in the model. These are generally the final output layers.

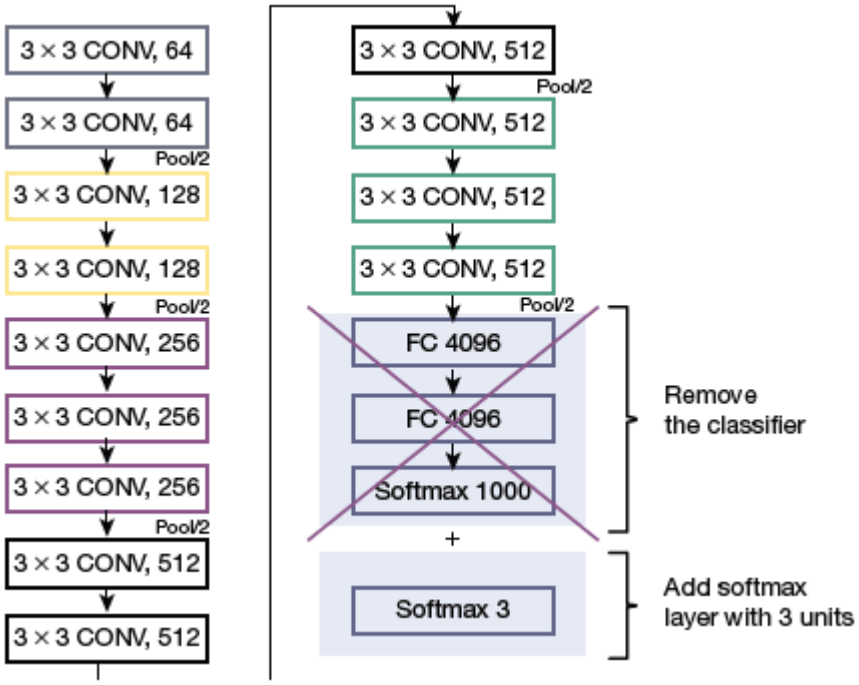


FIGURE 10.43 Fine Tuning the model

Step 5: Train the new layers. The pre-trained model's final output may be different from that of our new model. For example, if a pre-trained model was trained on the ImageNet dataset to generate 1000 classes and the new task at hand expects the output to be having just two classes, then we have to train the model with a new output layer in place.

Step 6: Fine-tune your model. In this step, we unfreeze some layers of the base model and train the entire model again on the whole dataset at a very low learning rate. This improves performance of the model on the new dataset while preventing overfitting issues (refer **Fig. 10.43**).

10.4.8 Types of Deep Transfer Learning

Domain Adaptation

It is done when the source and target domains have different feature spaces and distributions. So, domain adaptation technique adapts (or alter) one or more source domains to transfer information to improve the performance of a target learner. The adaptation of source domain is pur-

posely done bring the distribution of the source closer to that of the target.

Domain Confusion

In a neural network, different layers identify different features. For best results, the source and target domains should be as similar as possible. The domain confusion technique creates confusion at the source domain to confuse the high-level classification layers of a neural network. The confusion is created by matching the distributions of the target and source domains to ensure that samples are indistinguishable to the classifier.

While using domain confusion, we need to minimize the classification loss for the source samples, and the domain confusion loss for all samples.

Multi-Task Learning

In this approach (refer **Fig. 10.44**), several tasks from the same domain are learned simultaneously without distinction between the source and targets. This helps to transfer knowledge from each scenario of the same domain. The shared knowledge helps the learner to optimize the learning (or performance) across all of the n tasks.

Multi-task Learning in Deep Transfer Learning

One-Shot Learning

In this learning, the classification task is given one or a few examples to learn from. After learning, the classifier then classifies many new examples in the future. For example, a face recognition algorithm learns a person's face that must be classified correctly with different facial expressions, lighting conditions, accessories and hairstyles. This model is fed with one or a few template photos as input.

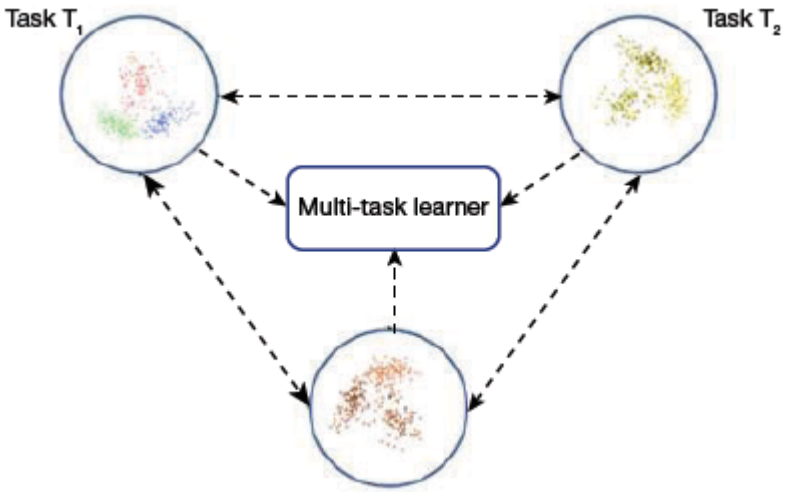


FIGURE 10.44 Multi-Task Learning

Zero-Shot Learning

In this learning technique, transfer learning is done with zero instances of a class. This learning does not depend on labelled data samples. But it does require additional data during the training phase to understand the unseen data.

Zero-shot learning focuses on the traditional input variable, *x*, the traditional output variable, *y*, and the task-specific random variable. It is extensively used in applications like machine translation, where we may not have labels in the target language.

10.4.9 Applications of Transfer Learning

1. **Learning from simulations:** Transfer learning is used when traditional machine learning applications that rely on hardware for interaction makes data gathering and model training an expensive and time-consuming or simply too dangerous affair.
   In such a scenario, simulation is the preferred tool. Transfer learning is done by applying the knowledge gained by simulation process to the real world. Though the feature spaces between source and target domain are the same (both generally rely on pixels), the marginal probability distributions are different, as objects in the simulation and the source *look* different. Even the conditional probability distributions between simulation and real world might be different as the simulation is not able to fully replicate all reactions in the real world.

Udacity has open-sourced the simulator it uses for teaching its self-driving car engineer nanodegree.

Learning from simulations has the following benefits:

1. Data gathering becomes easy as objects can be easily bounded and analysed.
2. Fosters training as learning can be parallelized across multiple instances.

Simulation is also important in robotics. Training models on a real robot consume a lot of time and other resources. So, a better alternative is to learn from a simulation and transfer the knowledge to a real-world robot. Figure 10.45 shows a self-driving car simulator.

Credit: Have a nice day Photo / Shutterstock

FIGURE 10.45 Self-driving car simulator

2. **Adapting to new domains:** We know that the data where labelled information is easily available and the data that we actually care about are different. Even if the training and the test data *look* the same, training data may have a bias that is unnoticeable to humans but the model may exploit to overfit on the training data.

   Domain adaptation is also used to adapt to different text types. For example, if an NLP model is trained, then it will have difficulty coping with more novel text forms such as social media messages.

   Recently, domain adaptation is also being used in automatic speech recognition (ASR) applications. Such an application may be trained with voice of 500 speakers, but only those people with a similar accent can be benefited. The rest may have trouble being understood.
3. **Transferring knowledge across languages:** Another important application of transfer learning is learning from one language and applying

that knowledge to another language. Reliable cross-lingual adaptation methods allow the vast amounts of labelled data in English and apply them to any language, particularly those that are truly low-resource languages.

10.4.10 When Does Transfer Learning *Not* Work?

Transfer learning should not be used when the weights trained from the source task are different from that of the target task. For example, if the model that was trained to identify tigers cannot be used to identify a sports car. Initializing a model with pre-trained weights that correspond with similar outputs is better than using weights with no correlation with the expected output.

Removing layers from a pre-trained model may cause issues with the architecture of the model. For example, if the first layer is removed, then the model will have a low learning rate as it has to juggle working with low-level features. Moreover, the model may suffer from overfitting issues as removing layers reduces the number of parameters that can be trained. Therefore, using a model with the correct number of layers is vital to the success of a project using transfer learning.



KEY TERMS

Population: It is a subset of all the possible solutions to the given problem.

Chromosomes: A chromosome is the individual solution to the given problem.

Gene: A gene is one element position of a chromosome.

Allele: It is the value which a gene takes for a particular chromosome.

***Genotype*:** Genotype is the population (solution) in the computation space that can be represented in a simple and easy-to-understand manner so that it can be easily manipulated using a computing system.

***Phenotype*:** Phenotype is the population (solution) in the real-world solution space that can be represented in real world situations.

***Fitness Function** (or* **Evaluation Function):** It is the function which accepts the solution as input and produces the suitability of the solution as the output.

Genetic Operators: Operators which change the genetic composition of the offspring are known as genetic operators. These include crossover, mutation, selection, etc.

Mutation: The process in which a new solution is obtained by introducing a small random tweak in the chromosome.

Parent selection: The process of selecting individuals from the population (or parents) which will mate and recombine to generate off-springs for the next generation.

Soft computing: A group of computational techniques that are based on artificial intelligence (AI) and natural selection. It is often used to develop cost-effective solutions for complex real-world problems that are either difficult or impossible to be solved by conventional computing techniques.

Transfer learning: The reuse of a pre-trained model on a new problem.



RECAPITULATION POINTS

1. A set is a collection of unordered or ordered elements. The classical set is defined in such a way that the universe of discourse (set of all possible values, also known as universal set) is divided into two groups—members and non-members.
2. A fuzzy set denoted by the tilde (~) character is a collection of values which exist between 0 and 1. In the fuzzy set, there is also a concept of partial membership.
3. In genetic algorithms, the given problem has a pool or a population of possible solutions. Crossover and mutation are applied on these solu-

tions to produce new children. This process is repeated over several generations.

4. While decoding is the process of transforming a solution from the genotype to phenotype space, encoding, on the other hand, transforms solution from the phenotype to genotype space.
5. The most popularly used technique for selecting parents is the fitness proportionate selection. In this technique, every individual can become a parent with a probability which is proportional to its fitness. This means that fitter individuals have a higher chance of mating and propagating their features to the next generation.
6. The survivor selection policy is used to determine which individuals will be a part of the next generation and which will be discarded.



REVIEW QUESTIONS

1. Differentiate between the following.
    1. Speech and voice recognition
    2. Classical set theory and fuzzy set theory
    3. Soft computing and hard computing
    4. Artificial intelligence and soft computing
2. Explain the architecture of a fuzzy logic system.
3. Discuss the different techniques of selecting parent in case of genetic algorithms.
4. Explain the crossover technique.
5. Mutation plays an important role in genetic algorithms as it maintains as well as introduces diversity in the genetic population. Justify this statement.
6. Write short notes on genetic algorithms.
7. Using order crossover, generate two children from the given information.
   Parent 1 = **1 2 3** | **4 5 6** 7 | **8 9**
   Parent 2 = **4 5 2** | **1 8** 7 **6** | **9 3**
8. With a help of an example, highlight the difference between fuzzy logic and Boolean logic.
9. 'Manya is looking for a **match**.' is lexically ambiguous. Comment.
10. When is it recommended to use transfer learning?

11. Explain the different approaches to transfer learning.
12. What techniques will be used for transfer learning in case of deep learning?
13. Differentiate between traditional machine learning and transfer learning.



FILL UP

1. _____ ambiguity usually happens when pronouns are used.
2. _____ occurs when we omit words that are actually needed for grammatical completion of the syntax.
3. _____ uses the most plausible interpretation is used.
4. _____ is a superset of regular grammar.
5. _____ is a compiler that breaks the data coming from lexical analysis phase into smaller elements.
6. _____ provides many values between the true and false.
7. _____ systems combine fuzzy logic and neurocomputing and induces rules from _____.
8. _____ is a subset of all the possible solutions to the given problem.
9. _____ is the population (solution) in the computation space.
10. _____ is the function which accepts the solution as input and produces the suitability of the solution as the output.
11. One extremely fit solution taking up of the entire population, results in a situation known as _____.
12. In _____ technique, more than one parent is selected and one or more off-springs are produced using the genetic material of the parents.
13. _____ transfer learning requires the source and target domains to be the same.
14. In _____ approach, several tasks from the same domain are learned simultaneously without distinction between the source and targets.



STATE TRUE OR FALSE

1. A set is a collection of only ordered elements.

2. Algorithms based on fuzzy logic produce accurate results even with imprecise or inaccurate data.
3. Fuzzy system should be used when it is difficult to map an input space to an output space.
4. Chromosomes is the value which a gene takes for a particular chromosome.
5. Genotype is the population (solution) in the real-world solution space that can be represented in real world situations.
6. Generational genetic algorithm generates ‘n’ off-springs in each iteration.
7. Crossover is the process in which a new solution is obtained by introducing a small random tweak in the chromosome.
8. Soft computing algorithms are not affected by changes in the environment.
9. Solutions given by soft computing are uncertain as we get different solutions on solving the same problem using the same set of input data at different intervals of time.
10. Referential ambiguity exists when you are referring to something using the pronoun.



MULTIPLE CHOICE QUESTIONS

1. Identify the incorrect statement about fuzzy system.
    1. Fuzzy logic works similarly as the human reasoning.
    2. Fuzzy set theory is based on the set theory of mathematics, so users can easily relate its basic concepts.
    3. Developing a system based on fuzzy logic takes more time than developing it using conventional methods.
    4. Fuzzy logic supports the concept of partial truth, such as 0.9 or 0.5.
2. Identify the incorrect statement about genetic algorithms.
    1. Can perform parallel computations.
    2. Optimize continuous as well as discrete functions.
    3. Give optimized solutions for multi-objective problems.
    4. Return only one solution that best performs in all scenarios.
3. A _____ is one element position of a chromosome.
    1. Population

2. Gene
3. DNA
4. Allele

4. _____ representation is used for problems in which the solution is represented by an order of elements
    1. Integer
    2. Real-valued
    3. Permutation
    4. Binary
5. _____ selection is used when the individuals in the population have very close fitness values.
    1. Rank
    2. Tournament
    3. Roulette wheel
    4. Stochastic universal sampling
6. _____ learning is the reuse of a pre-trained model on a new problem.
    1. Transfer
    2. Machine
    3. Deep
    4. Virtual
7. Transfer learning gives better performance when there is a higher _____.
    1. Start
    2. Slope
    3. Asymptote
    4. All of these
8. Identify the incorrect statement.
    1. Metaverse uses digital currencies and nonfungible tokens (NFTs).
    2. Transfer learning is a machine learning technique.
    3. In transfer learning, we use the early and middle layers of the model and retrain the latter layers.
    4. If input for two tasks is the same, then we should re-use the model.
9. _____ provides numerous pre-trained models for transfer learning.
    1. Scikit
    2. numpy
    3. pandas
    4. keras

10. _____ is done when the source and target domains have different feature spaces and distributions
    1. Domain adaptation
    2. Domain confusion
    3. Multi-task learning
    4. One-shot learning
11. In applications like machine translation, _____ type of deep learning is used.
    1. Domain adaptation
    2. Zero-shot learning
    3. Multi-task learning
    4. One-shot learning



ANSWERS

Fill Up

1. Referential
2. Ellipsis
3. *World knowledge*
4. Context-free grammar
5. Parser
6. Fuzzy logic
7. Neuro-fuzzy, observations
8. Population
9. Genotype
10. Fitness function
11. premature convergence
12. Crossover
13. Inductive
14. Multi-task learning

State True or False

1. False
2. True
3. False

4. False
5. False
6. True
7. False
8. True
9. True
10. True

Multiple Choice Questions

1. c
2. d
3. b
4. c
5. a
6. a
7. d
8. b
9. d
10. a
11. b